Journal of Network and Computer Applications 242 (2025) 104255



Contents lists available at ScienceDirect

Journal of Network and Computer Applications

journal homepage: www.elsevier.com/locate/jnca



Research paper



Exploring transferable adversarial attacks for Deep Learning-based Network Intrusion Detection☆

Zhongshu Mao [a], Yiqin Lu [a,*], Zhe Cheng [b], Kaiqiong Chen [a]

[a] School of Electronic and Information Engineering, South China University of Technology, Guangzhou, 510641, China
[b] School of Computer Science & Engineering, South China University of Technology, Guangzhou, 510641, China

ARTICLE INFO

Keywords:
NIDS
Deep learning
Adversarial attack
Transferability

ABSTRACT

Network Intrusion Detection Systems (NIDSs) increasingly use Deep Learning (DL) techniques due to their superior performance. However, some studies have shown that attackers can bypass DL-based NIDSs by generating Adversarial Attack Traffic (AAT). To better understand the vulnerabilities of DL-based NIDS, more and more adversarial attacks have been proposed. We observed three problems while studying these attacks: (1) Some attacks need to query the target model to construct AAT or surrogate models, which is not stealthy enough; (2) The generated AAT is impractical due to the lack of constraints when modifying features; and (3) The attack methods are limited in their extensibility. We propose a framework called SPTS to address these problems. SPTS runs in the black-box scenario without access to the target model. To generate the practical AAT, SPTS incorporates feature hierarchization and rectification. The correlations and constraints between features are established by mathematics. In addition, we implement a variety of adversarial attack algorithms within the SPTS framework, illustrating its excellent scalability. The AAT is mapped to practical packets to evaluate its transferability. Furthermore, we discover that enhancing the diversity of gradients can further improve the transferability of AAT. We propose a DGM algorithm based on SPTS, which randomly transforms the inputs to produce more robust gradients. Empirical evaluations on the standard dataset demonstrate the effectiveness and superiority of our SPTS and DGM. Defense methods to mitigate SPTS and DGM are also provided, and their advantages and disadvantages are described based on experimental results. Code is available at https://github.com/maozhongshu1995/TransAdvAttForNIDS.

1. Introduction

Network Intrusion Detection Systems (NIDSs) monitor and analyze various types of network traffic (e.g., cloud computing Keegan et al. (2016), Internet of Things (IoT) Frustaci et al. (2017), edge computing Chen et al. (2019), etc.) to detect potential malicious activities and intrusions at an early stage. They are important components of modern network security systems. Compared to traditional machine learning techniques, Deep Learning-based (DL) NIDSs have shown strong performance and can eliminate feature extraction and reduction processes (He et al., 2023). However, DL-based NIDSs have been shown to be vulnerable to adversarial attacks (Wang, 2018). An attacker can manipulate the specified features of the raw attack traffic, e.g., sending delay and number of packets (Hashemi et al., 2019), to generate Adversarial Attack Traffic (AAT). The AAT can misclassify the target NIDSs, i.e., evade the detection of NIDSs. If such attacks are ignored, DL-based NIDSs will become useless. Therefore, more and more researchers are focusing on adversarial attacks to better understand the vulnerabilities of DL-based NIDSs.

Based on the knowledge possessed by the attacker, adversarial attacks are also categorized as white-box, gray-box, and black-box attacks (He et al., 2023). In white-box attacks, the attacker knows all the information about the target model, including the model structure, parameters, and other details. In black-box attacks, the attacker considers the target model as a black box and has no knowledge about it. Gray-box attacks are between white-box and black-box attacks, e.g., only the structure of the model is known, but not the parameters. Intuitively, gray-box and black-box attacks are more threatening and closer to realistic attack scenarios. Because it is usually not trivial for an attacker to get crucial information about the target NIDSs. In this paper, we focus on the black-box attack, where the attacker has no prior knowledge about the target NIDSs.

While studying existing adversarial attacks, we observe three problems. First, some attacks rely on querying the target model to construct

☆ This work is supported by the Research and Development Program in Key Areas of Guangdong Province, China, No. 2019B010137001.
* Corresponding author.
E-mail address: qin@scut.edu.cn (Y. Lu).

https://doi.org/10.1016/j.jnca.2025.104255
Received 16 December 2024; Received in revised form 31 May 2025; Accepted 18 June 2025
Available online 30 June 2025
1084-8045/© 2025 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

AAT or surrogate models, which is not stealthy enough. Kuppa et al. (2019) and Yan et al. (2023) need to access target NIDSs to build the local surrogate models and then use these models to generate AAT. On the other hand, Zhang et al. (2020), Alhussien et al. (2024), Sun et al. (2025) and Okada et al. (2025) craft AAT by directly accessing the target NIDSs. Excessive access to the target NIDSs will easily attract the defender, which may lead to the failure of the attack. Second, there are no constraints or only limited constraints on feature modification, resulting in the impractical AAT. Images are unstructured, while the traffic is structured. If the traffic is treated as images without considering the correlation and constraints between the features, the practical AAT cannot be generated, e.g., Yang et al. (2018), Zhang et al. (2022), Roshan and Zafar (2024), Adeke et al. (2025), Liu et al. (2025) and Zhang et al. (2025). Third, the attack methods are limited in their extensibility. Sharon et al. (2022) only modify the interval between sending packets, Debicha et al. (2023) create AAT based on the mean difference method, and Zhang et al. (2024) investigate the transferability of AAT for Autoencoder-based NIDSs. Hore et al. (2025) and Lu et al. (2025) are also capable of generating practical AAT. Although these methods can produce the practical AAT, they have their own unique frameworks and are challenging to extend.

We propose a framework called Strolling in Practical Traffic Space (SPTS) to address the above problems. SPTS generates the AAT with high transferability, which means that an adversarial example made for one model can be misclassified by another model (Liu et al., 2017). To generate the practical AAT, SPTS involves feature hierarchization and rectification. This not only distinguishes different features but also indicates their correlation and constraints. For example, level 2 features must be computed from level 0 and level 1 features, where level 0 features are non-perturbable and level 1 features are perturbable. We establish these correlations and constraints between features by mathematics to ensure that the generated AAT is practical. The AAT is mapped to practical packets to evaluate its transferability. In addition, SPTS is highly extensible and can implement various adversarial attacks. We can evaluate different attacks within the SPTS framework with more objectivity and fairness.

It is low-cost to study AAT generation methods based on SPTS. Inspired by model augmentation (Lin et al., 2019; Xie et al., 2019), we find that enhancing the gradient diversity can further improve the transferability of AAT and propose the Diversity Gradient Method (DGM). Specifically, DGM is based on SPTS and uses randomized transformations on the input to produce more robust gradients. We validate the effectiveness of DGM across different Deep Neural Networks (DNNs), including Multilayer Perceptron (MLP) (Faker and Dogdu, 2019), Convolutional Neural Network (CNN) (Zhang et al., 2019a), ResCNN (i.e., CNN with residual connection He et al. (2016)), LSTM (Zhang et al., 2019b), and Self-attention (i.e., models based on self-attention mechanism Vaswani et al. (2017)). Empirical evaluations on the standard dataset demonstrate the effectiveness and superiority of our SPTS and DGM.

Finally, we provide two defense methods to mitigate SPTS and DGM: normal adversarial training and adversarial training with SPTS. The former treats the traffic as unstructured data when training the model, while the latter depends on the correlations and constraints between the features. Although adversarial training can help NIDS detect AAT, the experimental results show several limitations remain. Our contributions are summarized as follows:

- We introduce a black-box attack framework called SPTS, which operates without accessing the target model and makes the adversarial attack more stealthy.
- To generate the practical AAT, SPTS involves feature hierarchization and rectification. The correlations and constraints between features are established by mathematics.
- SPTS is highly extensible. Various adversarial attacks are implemented based on the SPTS framework and evaluated with objectivity and fairness.
- We find that enhancing the gradient diversity can further improve the transferability of AAT and propose the DGM based on SPTS.

The rest of the article is organized as follows. Section 2 presents the background and related work. Section 3 covers the threat model. Section 4 describes SPTS and DGM. Section 5 contains experiments and analysis. Limitations are discussed in Section 6. Section 7 summarizes the article.

2. Background and related work

2.1. Network intrusion detection systems

NIDSs are a key topic in cybersecurity (Jmila and Khedher, 2022), which can monitor and analyze traffic in the network to detect potential malicious activities and intrusions at an early stage. There are usually two approaches to implementing NIDSs, namely signature-based and anomaly-based approaches (Sharon et al., 2022). The signature-based approach makes use of predefined attack signatures, while the anomaly-based approach detects unusual patterns by analyzing traffic behavior. Anomaly-based NIDSs can recognize attacks that are not seen (such as zero-day attacks He et al. (2023)) and are more interesting for research, compared to Signature-based NIDSs. This paper focuses on anomaly-based NIDSs (all NIDSs in this study refer to anomaly-based approaches unless otherwise noted).

DNNs are increasingly used for NID purposes, as they help minimize feature engineering efforts and operate with high detection accuracy (Zhang et al., 2022). DL-based NIDSs can easily capture non-linear relationships between traffic and learn behavioral patterns of normal and abnormal traffic, leading to better generalization to unseen traffic. However, DL-based NIDSs have been studied less for their robustness against practical AAT, and therefore, this work focuses on DL-based NIDSs.

2.2. Adversarial attack

Adversarial attacks (Goodfellow et al., 2014) are carefully designed modifications of the input data to deceive the target (classification) model into misjudging or misclassifying it. These modified data are known as adversarial examples. Adversarial attacks can be categorized into white-box, gray-box, and black-box attacks (He et al., 2023), depending on the knowledge possessed by the attacker (as present in Section 1). This paper specifically focuses on black-box attacks. In Computer Vision (CV), the method based on the surrogate model has been widely researched for black-box attacks, e.g., Xie et al. (2019), Lin et al. (2019), Dong et al. (2019) and Dong et al. (2018). The attacker first needs to train a local surrogate model which behaves similarly to the target model. Then, they can craft adversarial examples on the surrogate model. This method depends on the transferability of the adversarial examples (Liu et al., 2017), i.e., the adversarial examples made for one model can misclassify another. There are two common ways to enhance transferability: model augmentation (also known as data augmentation) and advanced optimization algorithms (Lin et al., 2019). Model augmentation transforms the input, which is equivalent to transforming the model, to produce more robust gradients, e.g., Xie et al. (2019), Lin et al. (2019) and Dong et al. (2019). Advanced optimization algorithms make use of sophisticated optimization algorithms to compute gradients, e.g., Dong et al. (2018), Wang and He (2021) and Jang et al. (2022).

2.3. Related work

Adversarial attacks can be categorized into white-box, gray-box, and black-box attacks. In white-box attacks, the attacker has full knowledge of the target model, making the attack relatively easy to launch. Wang (2018) evaluate the performance of DL-based NIDSs using FGSM. Warzyński and Kołaczek (2018) verify the resilience of MLP-based NIDSs against adversarial attacks. Hashemi et al. (2019) assess the existing NIDSs in adversarial settings. Ibitoye et al. (2019) compare the efficiency of FNN and SNN in an IoT environment. Abou Khamis and Matrawy (2020) focus on the efficiency of various white-box evasion attacks. Mohammadian et al. (2023) propose a new attack targeting DL-based NIDSs, which perturbs only three features with a perturbation magnitude of 0.1. In gray-box attacks, the attacker possesses only partial knowledge of the target model, making it more challenging than white-box attacks. Apruzzese et al. (2019) analyze realistic adversarial attacks in a gray-box scenario. Qiu et al. (2020) assume that the attacker knows the mechanism of the feature extractor, generating ATT based on the saliency map. Lin et al. (2022) generate adversarial features with malicious features and some noise.

In black-box attacks, the attacker has no knowledge of the target model, making it extremely challenging. To address this issue, researchers have proposed various methods for generating AAT. These methods can be broadly classified into four categories. The first category mainly includes approaches that utilize generative models, such as Generative Adversarial Networks (GANs). Alhajjar et al. (2021) and Han et al. (2021) combine the generative models with evolutionary computation to produce AAT. Wang et al. (2024) utilize constraints and DoppelGANger to generate AAT. Du et al. (2025) introduce a transformer-based framework that includes generating reversible metadata feature vectors. Yao et al. (2025) also examine the vectorization and recovery between vectors and raw traffic, constructing AAT based on PS-WGAN. Since this study focuses on non-generative model-based approaches, we limit our discussion of these methods.

The second category mainly consists of query-based methods, where the attacker requires access to the outputs of the target NIDS in order to construct a surrogate model or generate AAT. Kuppa et al. (2019) introduce a new black-box attack in query constraint settings. Zhang et al. (2020) argue that the perturbation does not need to be small but cannot destroy the original functionality. Thus, they propose a brute-force black-box method based on the confidence scores of the target classifiers. Yan et al. (2023) explore a more realistic label-only black-box scenario. Alhussien et al. (2024) propose a set of domain constraints to preserve the statistical and semantic relationships between traffic features. Sun et al. (2025) conduct content-based attacks based on Large Language Model (LLM) and reinforcement learning (RL). In a black-box scenario, Okada et al. (2025) demonstrate the importance of features across various attack types using KernelSHAP, which guides the modification of raw packets. Nevertheless, an excessive number of queries may alert the victims and compromise the success of the attack.

The third category primarily focuses on feature-level attacks. Yang et al. (2018) evaluate the performance of various adversarial attacks in a black-box scenario. Zhang et al. (2022) introduce a framework called TIKI-TAKA to assess the robustness of DL-based NIDS. Meanwhile, Roshan and Zafar (2024) examine the black-box adversarial transferability phenomena using FGSM. Adeke et al. (2025) investigate the impact of feature selection on adversarial transferability of the ZOO method. Liu et al. (2025) conduct a study of adversarial attacks on RNN models with time steps in NIDS. Zhang et al. (2025) introduce an explainable transfer-based black-Box adversarial attack framework. Nevertheless, there are no constraints or only limited constraints on feature modification, resulting in the impractical AAT.

The remaining approaches fall into the fourth category, where most studies focus on enhancing the transferability of AAT through surrogate models. Sharon et al. (2022) use LSTM to adjust the sending delay of packets so that the time distribution of the attack traffic approximates the normal traffic. Debicha et al. (2023) fabricate AAT based on the mean and variance of the normal and abnormal traffic. Zhang et al. (2024) propose a linear autoencoder as the surrogate model to study the transferability of autoencoder-based NIDSs. Hore et al. (2025) convert each packet into normalized numeric feature values to construct AAT. Lu et al. (2025) introduce a time series-based adversarial attack framework to perturb the inter-arrival time and flow duration. Although these methods can produce the practical AAT, they have their own unique frameworks and are challenging to extend. As shown in Table 1, we summarize the mentioned black-box attacks to clearly show the difference between our work and previous works. It should be emphasized that the approach presented in this study belongs to the fourth category and is dedicated to improving the transferability of AAT based on surrogate models.

3. Threat model

This section will define the threat model from three aspects: the attacker's knowledge, capability, and target. First, we assume that the attacker has no knowledge of the architecture and training data of the target NIDSs, i.e., NIDSs are treated as a black box. However, due to the public availability of traffic collection and analysis tools such as tcpdump,[1] Tstat,[2] Wireshark,[3] CICFlowMeter,[4] etc., we assume that the attacker is able to know the majority of the characteristics of NIDSs. This assumption is similar to the threat model in Wang et al. (2024), Debicha et al. (2023) and Zhang et al. (2024), which guarantees that the threat model approximates a real attack scenario. Nevertheless, some features are essential to the attacker because of the unique characteristics of our proposed approach. A detailed explanation will be provided in Section 5.4.4. Second, we follow previous studies (Wang et al., 2024; Sharon et al., 2022; Debicha et al., 2023; Zhang et al., 2024) and assume that the attacker is able to access the network where the target NIDSs are located and collect normal traffic. The attacker can train local surrogate models based on the collected traffic to simulate the behavior of NIDSs. Furthermore, we assume that the attacker has no access to NIDSs or their output because too much access to NIDSs may draw the victim's attention and cause the attack to fail. Finally, there are two goals for the attacker: (1) creating AAT to evade the detection of NIDSs and (2) ensuring that the created AAT is practical.

4. Attack methodology

4.1. Strolling in practical traffic space

Generating AAT by directly modifying the traffic features is a very simple task, as noted in Qiu et al. (2020), Zhang et al. (2022) and Roshan and Zafar (2024). However, distinguishing whether the generated AAT is practical presents a challenge. One straightforward way is to check whether the AAT can be mapped to packets. If the mapping is feasible, then the generated AAT is practical; otherwise, it is impractical. Unfortunately, we discover that this mapping is very hard. Consider a scenario where we have raw attack traffic $X = [4, 2, 4, 3, 3]$, represented by five dimensions (i.e., five features): the number of packets, the minimum, maximum, and mean packet length, and the packet length variance, respectively. Let us assume a small perturbation, $\epsilon = [1, -1, -1, -1, -1]$. Then the generated AAT $X' = X + \epsilon$, i.e., $X' = [5, 1, 3, 2, 2]$. Obviously, it is hard to map X' to individual packets. Furthermore, the difficulty of the mapping will rise as the increment of the dimensions of X and ϵ.

However, the mapping becomes easier if the dimension of the perturbation is reduced. In this case, the perturbation $\epsilon = [0, 0, 1, 0, 0]$,

[1] www.tcpdump.org/.
[2] tstat.polito.it/.
[3] www.wireshark.org/.
[4] www.unb.ca/cic/research/applications.html#CICFlowMeter.

Table 1
A summary of black-box attacks.

Author	Main method and model	Modified features	Need access to target NIDS	Is AAT practical	Target NIDS
Alhajjar et al. (2021)	GAN, PSO, GA	29 mutable features	No	✓	SVM, DT, NB, KNN, RF, MLP, GB, LR, LDA, QDA, BAG
Han et al. (2021)	GAN, PSO	Interarrival time of packets, Protocol, Payload size	No	✓	KitNET, LR, DT, SVM, MLP, IF
Wang et al. (2024)	DoppelGANger, Constraint	N/A	No	✓	KNN, RF, XGboost, CNN, LSTM, Transformer
Du et al. (2025)	Transformer, RFPMPSO, PPO	Packet size, count, jitter and TTL	No	✓	Kitsune, LSTM, RNN-IDS, DNN-IDS, FlowTransformer, GNN-IDS
Yao et al. (2025)	Reversible traffic vectorization, PS-WGAN, DNN	Inter-arrival time, Protocol layer, Payload size	No	✓	CNN, GRU, LSTM, CNN-GRU, CNN-LSTM, Bidirectional GRU, LSTM, and Kitsune
Kuppa et al. (2019)	MAA, Spherical adversarial subspaces	Time based features	Yes	✓	AE, OC-SVM, IF, DSVDD, DAGMM, AnoGAN, ALAD
Zhang et al. (2020)	Brute-force Modification	All nonfunctional features	Yes	✓	LR, DT, MLP, NB, RF
Yan et al. (2023)	PGD, Optimization problem	Time-related, Length-related features, Increasing packet number	Yes	✓	LR, SVM, NB, DT, RF, XGboost, NN
Alhussien et al. (2024)	Domain Constraints, ZOO, DeepFool, C&W	Mutable features including srcbytes, dstbytes, duration and port numbers	Yes	Both	DT, DNN
Sun et al. (2025)	DL models, LLM, RL	Non-functional fields	Yes	✓	MalTraffic, RBRN, DeepMal, TextCNN, DeepLog, BERT
Okada et al. (2025)	KernelSHAP	Features varies across different types of attacks.	Yes	✓	DL-NIDS
Yang et al. (2018)	WGAN, ZOO, C&W	All features	Both	✗	NB, RF, SVM, DNN
Zhang et al. (2022)	NES, Boundary, Pointwise, HopSkipJumpAttack, OPT-attack	22 time-based features	Yes	✗	MLP, CNN, C-LSTM
Roshan and Zafar (2024)	FGSM	All features	No	✗	DNN
Adeke et al. (2025)	ZOO, Correlation coefficient, Mutual information, Chi-square, Mean absolute difference	All features	Yes	✗	RF, DT, SVM, LR, DNNs, DBN
Liu et al. (2025)	AutoEncoder, TD-RNN	Non-functional features	No	✗	ORNN, LSTM, GRU
Zhang et al. (2025)	ISFS, CoGT, ensemble models, ZO-SGD	Non-robust features	No	✗	MLP, AlertNet, IDSNet, DeepNet, RF, Xgboost, MaMPF, FS-Net, KitNet, Diff-RF
Sharon et al. (2022)	LSTM	Timestamp	No	✓	AE, KitNET, IF
Debicha et al. (2023)	Traffic mean value	Duration, TotPkts, OutBytes, InBytes	No	✓	MLP, RF, KNN
Zhang et al. (2024)	Autoencoder, R-FGSM, T-BIM	All continuous features	No	✓	DAE, DAGMM, VAE, SAE, LAKE
Hore et al. (2025)	Ensemble model, DDQN	Fragmentation, TTL, Window size, Maximum segment size, Window scale, Segment information	No	✓	DT, RF, MLP, DNN, SVM
Lu et al. (2025)	FGSM, UAP, 1D-CNN	Inter-Arrival Time of partial packets, Flow Duration	Both	✓	1D-CNN, SDAE
Ours	Traffic transformations, Advanced optimization algorithms	Minimum and Maximum length of packets, Minimum and Maximum delay	No	✓	MLP, CNN, ResCNN, LSTM, Self-attention

meaning only the maximum packet length will be modified. The resulting AAT $X' = [4, 2, 5, 3, 3]$. Intuitively, the mapping appears to be considerably simpler since only the maximum packet length is modified. It should be mentioned that mapping X' to packets is still a bit challenging. The main reason is that the corresponding mean packet length and packet length variance are not **rectified**, as the maximum packet length changes. "Rectify" refers to correcting features based on the perturbed features, e.g., recalculating new values of the mean packet length and packet length variance based on the perturbed maximum packet length. Obviously, feature rectification is a crucial step for generating practical AAT.

From the above example, we can identify two essential principles for generating practical AAT. First, it is better to use as few perturbation dimensions as possible. Because lower dimensions make it easier to map the generated AAT to packets. Second, the perturbed features must be carefully selected to guarantee that other features are rectifiable. Because randomly choosing perturbed features can make it difficult to rectify other features, even with low perturbation dimensions. After derivations and calculations, we found that perturbing only the minimum and maximum packet length is a good choice. It contains only two dimensions, aligning with the first principle. Meanwhile, the number of packets does not need rectification since only the minimum and maximum packet lengths are perturbed. The average packet length and the packet length variance can be recalculated from other features, which aligns with the second principle. It should be stated that deriving formulas for rectifying the packet length variance is a bit challenging, and we will explain this in detail later. Similarly, if "length" is replaced by "delay" in the above example, perturbing the minimum and maximum inter-packet delay is also a wise decision. As a result, our proposed SPTS framework involves a total of 4 perturbations: minimum and maximum length of packets in the forward direction and minimum and maximum time between two packets sent in the forward direction.

Indeed, we have categorized all the features into three types: non-perturbable, perturbable, and rectifiable features. To make feature rectification more transparent and intuitive, we divide the rectifiable features into two classes in order of priority (referring to Section 4.1.1). In summary, we hierarchize all the features of the dataset into 4 levels in SPTS. Apart from the feature hierarchization and rectification, the complete SPTS includes training the surrogate model and attack processes. Fig. 1 illustrates the entire SPTS. An attacker begins by training local surrogate models using normal traffic collected from the target network. Then, it creates perturbations following adversarial attack algorithms, which are based on model augmentation or advanced optimization algorithms. The practical AAT is achieved by adding the raw attack traffic with the perturbations and rectifying them level by level. This process is repeated until the termination condition, such as reaching the maximum number of iterations, resulting in the practical AAT.

4.1.1. Feature hierarchization

SPTS limits the perturbations to 4 dimensions: minimum and maximum length of packets in the forward direction and minimum and maximum time between two packets sent in the forward direction. The 4 features are called level 1 features. We refer to those features rectified by level 1 features as level 2 features, e.g., $Fwd\ Pkt\ Len\ Mean = Fwd\ Pkt\ Len\ Mean + (\delta_1 + \delta_2)/(Tot\ Fwd\ Pkts)$, where $Fwd\ Pkt\ Len\ Mean$ denotes the mean packet length in the forward direction, $Tot\ Fwd\ Pkts$ denotes the total packets in the forward direction, δ_1 and δ_2 are perturbations of the minimum and maximum length of packets in the forward direction, respectively. Some features need to be rectified by level 2 features, for example, $Flow\ Pkts/s = (Tot\ Fwd\ Pkts + Tot\ Bwd\ Pkts)/Flow\ Duration$, where $Flow\ Pkts/s$ denotes the number of packets per second for the flow, $Tot\ Bwd\ Pkts$ denotes the total packets in the backward direction, and $Flow\ Duration$ denotes duration of the flow and is a level 2 feature. We refer to these features that need to be corrected by level 2 features as level 3 features. Features that never need to be rectified are called level 0 features, e.g., $Tot\ Fwd\ Pkts$. We utilize a total of 66 features and hierarchize them one by one (referring to Section 5.2 for more details). Table 2 lists all level 2 and 3 features. Features not listed are level 0 features, except for level 1 features.

4.1.2. Feature rectification

As shown in Fig. 1, the process of feature rectification is not complicated and can be conducted level by level. This section presents the details of feature rectification. First, there is no perturbation or

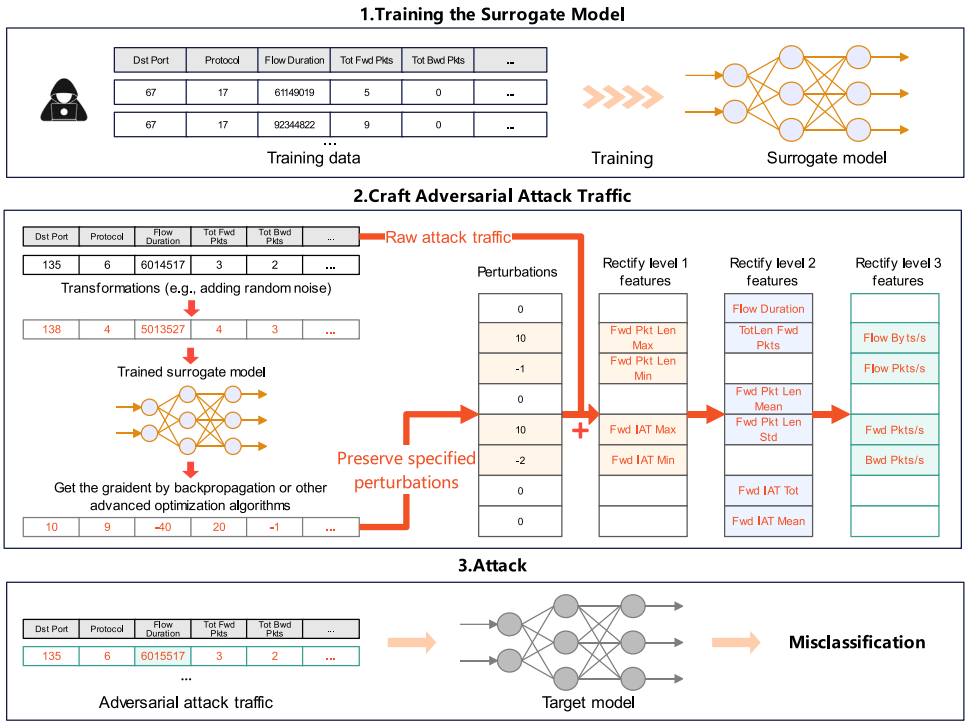


Fig. 1. Overview of SPTS. The framework is divided into three parts. In part 1, the attacker trains the surrogate model. In part 2, the attacker applies the attack algorithm to create AAT based on the surrogate model. In part 3, the attacker attacks the target NIDSs with the generated AAT.

Table 2
All level 2 and 3 features, and their rectification. δ_1 and δ_2 denote perturbations of the minimum and maximum length of packets in the forward direction. δ_3 and δ_4 denote perturbations of the minimum and maximum time between two packets sent in the forward direction.

Level	Features	Rectifying
Level 2	Flow duration	$+\delta_3+\delta_4$
	TotLen Fwd Pkts	$+\delta_1+\delta_2$
	Fwd Pkt Len Mean	$+(\delta_1+\delta_2)/(Tot\ Fwd\ Pkts)$
	Fwd Pkt Len Std	Calculated by Eqs. (1) and (2)
	Fwd IAT Tot	$+\delta_3+\delta_4$
	Fwd IAT Mean	$+(\delta_3+\delta_4)/(Tot\ Fwd\ Pkts-1)$
	Fwd IAT Std	Calculated by Eqs. (1) and (2)
	Pkt Len Min	$=min(Fwd\ Pkt\ Len\ Min, Bwd\ Pkt\ Len\ Min)$
	Pkt Len Max	$=max(Fwd\ Pkt\ Len\ Max, Bwd\ Pkt\ Len\ Max)$
	Pkt Len Mean	$+(\delta_1+\delta_2)/(Tot\ Fwd\ Pkts+Tot\ Bwd\ Pkts)$
	Pkt Len Std	Calculated by Eqs. (1) and (2)
	Pkt Len Var	Calculated by Eq. (1)
	Pkt Size Avg	$+(\delta_1+\delta_2)/(Tot\ Fwd\ Pkts+Tot\ Bwd\ Pkts)$
Level 3	Flow Byts/s	$=(TotLen\ Fwd\ Pkts+TotLen\ Bwd\ Pkts)/Flow\ Duration$
	Flow Pkts/s	$=(Tot\ Fwd\ Pkts+Tot\ Bwd\ Pkts)/Flow\ Duration$
	Fwd Pkts/s	$=Tot\ Fwd\ Pkts/Flow\ Duration$
	Bwd Pkts/s	$=Tot\ Bwd\ Pkts/Flow\ Duration$
	Subflow Fwd Byts	$=TotLen\ Fwd\ Pkts$

rectification to level 0 features. Second, for level 1 features, the rectification process involves adding the level 1 features with the generated perturbation (for convenience, this process is also referred to as rectification). However, while generating the perturbation, two aspects need consideration: the direction and size of the perturbation. For the direction, if we allow the minimum to increase or the maximum to decrease, it will create some logical contradictions. Briefly, it is possible for the minimum value to cease to be the minimum and the maximum value to cease to be the maximum. So, we permit only the maximum to increase and the minimum to decrease. For the size, the maximum and minimum values should be within a reasonable range; otherwise, the generated AAT may be impractical. In this paper, the minimum and maximum length of a packet in the forward direction are 0 and 1460, respectively. The minimum time between two packets sent in the forward direction is 1. There is no restriction to the maximum time between two packets sent in the forward direction. Because according to the maximum number of iterations and step size (referring to Section 5.1.3 for more details), the maximum time between two packets sent in the forward direction can only be increased by up to 980 μs, which will not lead to the connection timeout.

Third, level 2 features are rectified by level 1 features, most of which are simple computational procedures. For instance, the calculation of $Fwd\ Pkt\ Len\ Mean$ is in Section 4.1.1. The features that are comparatively challenging to compute are those standard deviation (or variance) related features. Actually, those features can be derived from the level 1 features since we only perturb the maximum and minimum values and do not change the number of packets. We abstract this problem into a mathematical one, considering n real numbers x_n ($n \geq 2$) with mean $\overline{x}$ and variance V, along with modification δ_1 and δ_2 for x_1 and x_2, respectively. The variance of the modified n real numbers:

$$V' = \frac{2}{n}(x_1\delta_1 - \overline{x}\delta_1 + x_2\delta_2 - \overline{x}\delta_2) + \frac{(\delta_1 - \overline{\delta})^2 + (\delta_2 - \overline{\delta})^2 + (n-2)\overline{\delta}^2}{n} + V \tag{1}$$

where $\overline{\delta}$ denotes the change in the mean and $\overline{\delta} = (\delta_1 + \delta_2)/n$. The derivation process is as follows:

$$V = \frac{1}{n}\sum_{i=1}^{n}(x_i - \overline{x})^2$$

$$V' = \frac{1}{n}(\sum_{i=1}^{2}(x_i - \overline{x} + \delta_i - \overline{\delta})^2 + \sum_{i=3}^{n}(x_i - \overline{x} - \overline{\delta})^2)$$

$$= \frac{1}{n}\sum_{i=1}^{2}((x_i - \overline{x})^2 + 2(x_i - \overline{x})(\delta_i - \overline{\delta}) + (\delta_i - \overline{\delta})^2) + \frac{1}{n}\sum_{i=3}^{n}((x_i - \overline{x})^2 - 2\overline{\delta}(x_i - \overline{x}) + \overline{\delta}^2)$$

Calculate $V' - V$:

$$V' - V = \frac{1}{n}\sum_{i=1}^{2}(2(x_i - \overline{x})(\delta_i - \overline{\delta}) + (\delta_i - \overline{\delta})^2) + \frac{1}{n}\sum_{i=3}^{n}(-2\overline{\delta}(x_i - \overline{x}) + \overline{\delta}^2)$$

Simplify $\sum_{i=3}^{n}(-2\overline{\delta}(x_i - \overline{x}) + \overline{\delta}^2)$:

$$\sum_{i=3}^{n}(-2\overline{\delta}(x_i - \overline{x}) + \overline{\delta}^2) = -2\overline{\delta}(\sum_{i=1}^{n}x_i - n\overline{x} - x_1 - x_2 + 2\overline{x}) + (n-2)\overline{\delta}^2 = -2\overline{\delta}(2\overline{x} - x_1 - x_2) + (n-2)\overline{\delta}^2 \tag{*}$$

Substitute (*) into $V' - V$ and end up with Eq. (1). Then, the standard deviation is given by:

$$Std' = \sqrt{V'} \tag{2}$$

Fourth, level 3 features are rectified by level 2 features, which are quite straightforward as with most level 2 features. Table 2 gives the rectification formulas for all level 2 features and level 3 features. It is important to note that if there are multiple identical maximum or minimum values, only one maximum and one minimum require adjustment.

4.1.3. The number of iterations and step size

Most processes of creating AAT are iterative, and so is SPTS. Some previous studies suggested stopping an iteration based on the feedback from surrogate models, i.e., ending the iteration when the surrogate model is misclassified. Nevertheless, the generated AAT could potentially overfit the surrogate model, just like a neural network overfitting the training dataset. Therefore, we use the predefined number of iterations and step size to prevent overfitting the surrogate model. Specifically, once the specified number of iterations has been reached, the current AAT will be considered the final output, regardless of whether the surrogate model misclassifies it.

4.2. Diversity gradient method

It is low-cost to study AAT generation methods based on SPTS, because it standardizes the process of feature rectification and allows us to concentrate more on perturbation generation. There are two common methods based on transferability: model augmentation and advanced optimization algorithms. Model augmentation, also known as data augmentation, is essentially a transformation of the input. Let x be the input, $Trans(x)$ be the transformation and f be the classification model. Then $f(Trans(x))$ denotes classifying the transformed input. If we denote $f(Trans(\cdot))$ as $f'(\cdot)$, f' could be seen as the enhanced f. DIM (Xie et al., 2019) is a classical model augmentation that primarily involves resizing and padding the input. Inspired by DIM, we randomly transform the inputs. This process increases the diversity of the gradients, resulting in more robust gradients and ultimately improving the transferability of AAT. Specifically, DGM creates N copies of the current data in each iteration, randomly applying transformations to each copy:

$$g = \nabla_{x_n} J(Trans(x_n), y) \tag{3}$$

where $J(\cdot)$ denotes the loss function, g denotes the gradient w.r.t. the input x_n and y denotes the label. x_n is the nth copy of x_t, where t is the number of iterations. $Trans(\cdot)$ denotes data augmentation methods, which specifically include the following four types:

$$Trans_1(x_n) = x_n \odot \nabla_{x_t} J(x_t, y)$$

where $\nabla_{x_t} J(x_t, y)$ denotes the gradient w.r.t. x_t at the tth iteration. $\odot$ denotes element-wise multiplication. This transformation is inspired by feature importance (Wang et al., 2021), which simply means amplifying the more important features.

$$Trans_2(x_n) = x_n + \gamma$$

where γ is a random perturbation with the same dimension as x_n and each dimension of γ ranges from 0 to 1.

$$Trans_3(x_n) = x_n \odot M$$

where M is a mask with the same dimension as x_n, made up of 0 and 1. This is just like the Dropout (Hinton et al., 2012), which can alleviate the overfitting. Each position of M is set to 1 with probability $1-p$ and 0 with probability p.

$$Trans_4(x_n) = x_n/(n+1)$$

This transformation is inspired by SIM (Lin et al., 2019), transforming the scale of the data. Their average gradient $\overline{g_t}$ is calculated after obtaining N gradients. Momentum is calculated by $\overline{g_t}$ and the current data is updated:

$$m_{t+1} = \mu \cdot m_t + \frac{\overline{g_t}}{\|\overline{g_t}\|_1} \tag{4}$$

$$x_{t+1} = x_t + l \cdot sign(m_{t+1}) \cdot mask \tag{5}$$

where m_t denotes the momentum at the tth iteration, μ denotes the momentum decay factor, and $sign(\cdot)$ denotes the sign function. $mask$ is a mask that serves to retain only the perturbations of level 1 features. l denotes the step size. Once x_{t+1} is obtained, features can be rectified to get the input for the next iteration. We provide the pseudo code for DGM within SPTS in Algorithm 1. One can modify lines 7 to 18 in Algorithm 1 to implement the MI-FGSM, SIM, and VMI-FGSM mentioned in Section 5.

5. Evaluation

5.1. Experimental setup

5.1.1. Dataset

The CIC-IDS-2018 (Sharafaldin et al., 2018) and TON_IoT (Alsaedi et al., 2020) datasets are used for all evaluations. The CIC-IDS-2018

Algorithm 1 DGM within SPTS

Input: A surrogate model f, a loss function J, raw attack traffic x with ground-truth label y, and a decay factor μ
Input: The number of iterations T, the step length l and the mask of level 1 features $mask$ for SPTS
Input: The dropout rate p and the number of copies N for DGM
Output: AAT x^{adv}
$x_0^{adv} = x$; $m_0 = 0$
for $t = 0 \rightarrow T$ **do**
  $g_{sum} = 0$
  Calculate the current gradient $\nabla_{x_t} J(x_t, y)$
  for $n = 0 \rightarrow N$ **do**
    Generate the dropout mask M
    Generate the random noise γ
    Copy the current data $x_n^{adv} = x_t^{adv}$
    Randomly select a data augmentation method and calculate the gradient g by Eq. (3)
    $g_{sum} = g_{sum} + g$
  end for
  $\overline{g_t} = g_{sum}/N$
  Update the momentum m_{t+1} by Eq. (4)
  Update the AAT x_{t+1}^{adv} by Eq. (5)
  Rectify x_{t+1}^{adv}
end for
$x^{adv} = x_T^{adv}$
return x^{adv}

is generated from the collaborative project between the CSE and the CIC. This dataset aims to provide real-world network traffic data for studying intrusion detection. The TON_IoT is constructed by UNSW Canberra IoT Labs and Cyber Range and contains telemetry data of IoT/IIoT services, operating systems logs, and network traffic of the IoT network. Note that only the network traffic data of TON_IoT is used because the purpose of this work is to generate practical AAT. The TON_IoT has provided preprocessed CSV files, which contain all the traffic. However, their features differ slightly from those in the CIC-IDS-2018. As a result, we extract all IP packets from raw PCAP files with scapy[5] and analyze them using CICFlowMeter.[6] In addition, In our threat model, the attacker does not have access to the training dataset of the target NIDSs. Therefore, we split each dataset into two sub-datasets for the target and surrogate models. Each sub-dataset is further divided into training and testing datasets, according to the ratio of 8 : 2. For convenience, the CIC-IDS-2018 and TON_IoT are abbreviated as IDS18 and TON in all tables, respectively. We recommend referring to Sharafaldin et al. (2018) and Alsaedi et al. (2020) for more details about the datasets.

5.1.2. DL-based NIDSs

Five DNNs are tested in this work, namely, Multilayer Perceptron (MLP) (Faker and Dogdu, 2019), Convolutional Neural Network (CNN) (Zhang et al., 2019a), ResCNN (i.e., CNN with residual connection He et al. (2016)), LSTM (Zhang et al., 2019b), and Self-attention (i.e., models based on self-attention mechanism Vaswani et al. (2017)). Thus, there are five target models and five surrogate models in all.

During designing the network architecture, we referred to the models used in Zhang et al. (2022). The architectures of these models have been adapted due to various experimental environments. MLP designed in this work contains 4 hidden layers with 256 units per layer and the activation function is ReLU. CNN consists of 6 convolutional blocks, each block contains a one-dimensional (1D) convolutional layer, a batch normalization layer, and an activation function layer (with LeakyReLU). The 1D convolutional layer has 64 kernels with a size of 5×5, a stride of 1, and a padding pattern of "*same*". The structure of ResCNN is identical to CNN, except for the connection between convolutional blocks. In particular, the input is added to the output as the input to the subsequent convolutional block, starting from the second convolutional block in ResCNN.

We consider a single traffic as a sequence in LSTM and Self-attention, and every 5 features as a word, i.e., the dimension of word embedding is 5. LSTM consists of 3 layers, each with 256 units. Self-attention is based on single-head attention, which captures the interaction between entities by Queries, Keys, and Values. Sine and cosine functions are used to calculate positional encodings. A fully connected feed-forward network (FFN) is followed. FFN is composed of 2 layers, the first layer has 100 units, and the second layer has 200 units. LeakyReLU is used as the activation function in FNN. Our self-attention-based model is designed without residual connection because of ResCNN. The final output layer of all models contains 2 units.

The only difference between the target and surrogate models is the number of neurons in the input layer (refer to Section 5.2 for more details). For convenience, the letters "t" and "s" are used to denote the target model and surrogate model, respectively. For example, MLP-t indicates the target model, and MLP-s indicates the surrogate model.

5.1.3. Baselines and attack settings

We chose MI-FGSM (Dong et al., 2018), VMI-FGSM (Wang and He, 2021), and SIM (Lin et al., 2019) as the baseline for SPTS and DGM. These algorithms are implemented within SPTS. The default settings of the algorithms are as follows unless otherwise specified. In SPTS, the number of iterations $T = 7$ and step size $l = 140$. The momentum decay factor $\mu = 1.0$ for all attacks. We set the sampling number $N = 5$ and $\beta = 1.5$ for VMI-FGSM and set the number of copies $m = 5$ for SIM. We set the sampling number $N = 5$ and dropout probability $p = 0.2$ for DGM.

5.2. Training NIDSs

The CICFlowMeter is used to extract 79 features from each traffic. Table 3 enumerates the 13 features removed, leaving 66 features to be used as input features for the target model. The *Timestamp* feature captures absolute time information, which is excluded to prevent the model from learning patterns of absolute time. This decision aligns with that of Okada et al. (2025) and Zhang et al. (2022). Among the remaining 12 features, most cannot be derived from other features through dependency relationships, and excluding them has a negligible impact on model performance. As shown in Table 5, NIDSs with 66 input features perform comparably to those using 78 input features. Consequently, these 12 features are also removed.

Table 3
Features dropped for target models. Surrogate models are trained without these features as well.

Features dropped	Timestamp	Active Std
	Flow IAT Mean	Active Max
	Flow IAT Std	Active Min
	Flow IAT Max	Idle Mean
	Flow IAT Min	Idle Std
	Active Mean	Idle Max
	Idle Min	

Table 4
Features dropped for surrogate models.

Features dropped	Bwd IAT Std	SYN Flag Cnt
	Fwd PSH Flags	Down/Up Ratio
	FIN Flag Cnt	Bwd Seg Size Avg

[5] https://scapy.net/.
[6] www.unb.ca/cic/research/applications.html#CICFlowMeter.

Table 5
The detection performance of all target and surrogate models.

Features number	Model	Accuracy (%)	Precision (%)	Recall (%)	F1 score (%)
		IDS18/TON	IDS18/TON	IDS18/TON	IDS18/TON
78	MLP-t	98.4/98.4	95.7/99.9	94.9/98.4	95.3/99.2
	CNN-t	98.5/98.6	96.2/99.9	94.9/98.6	95.5/99.3
	ResCNN-t	98.6/98.2	96.5/99.9	94.8/98.2	95.7/99.1
	LSTM-t	98.6/98.6	96.8/99.9	94.9/98.6	95.8/99.2
	Self-attention-t	97.9/98.5	92.7/99.9	94.9/98.6	93.8/99.2
66	MLP-t	98.7/98.6	97.1/99.9	94.8/98.6	96.0/99.3
	CNN-t	98.6/98.4	96.7/99.9	94.9/98.4	95.8/99.2
	ResCNN-t	98.6/98.2	96.7/99.9	94.8/98.2	95.7/99.1
	LSTM-t	98.7/98.7	97.3/99.9	94.9/98.7	96.1/99.3
	Self-attention-t	98.6/98.2	96.7/99.9	94.9/98.3	95.7/99.1
60	MLP-s	98.7/98.6	97.2/99.9	94.9/98.6	96.0/99.3
	CNN-s	98.6/98.5	97.0/99.9	94.9/98.5	95.9/99.2
	ResCNN-s	98.6/98.5	96.8/99.9	94.9/98.5	95.9/99.2
	LSTM-s	98.7/98.6	97.2/99.9	94.9/98.6	96.0/99.3
	Self-attention-s	98.6/98.3	96.7/99.9	95.0/98.4	95.8/99.1

As mentioned in Section 3, we argue that the attacker has knowledge of some input features of the target model and can infer some features. To simulate this situation, we randomly select some features of the target model to serve as the features inferred by the attacker. In particular, six extra level 0 features are randomly removed, and the remaining 60 features are used as input features for the surrogate model. Table 4 lists the 6 features removed for the surrogate model. The selection of both the number and specific features to be removed is performed randomly, which will be further discussed in detail in Section 5.4.4. Further, the data is padded when training the LSTM and Self-attention. Specifically, the length is padded up to a length of 80 with 0.

Random over-sampling is employed to address the class imbalance problem. We normalize the data using Min-Max (i.e., target models and surrogate models are normalized using the minimum and maximum values in their own training datasets). The batch size equals to 128. All models use the Adam optimizer with $\beta_1 = 0.99$, $\beta_2 = 0.99$, and the learning rate $lr = 0.001$. We train for 10 epochs using Cross Entropy as the loss function. As shown in Table 5, the performance of the target and surrogate models is indicated by accuracy, precision, recall, and F1 score. Both of the trained NIDSs exhibit excellent performance.

5.3. Adversarial attacks against NIDSs

5.3.1. Adversarial attack traffic

We consider the attack traffic in the test dataset of the surrogate model as raw attack traffic and ignore traffic with less than three packets in the forward direction. Because the traffic with the number of forward packets greater than three guarantees that at least two inter-packet delays in the forward direction and satisfies the condition of Eq. (1). For the CIC-IDS-2018, we use 84,688 pieces of raw attack traffic; detail is provided in Table 6. For the TON_IoT, we use 578,272 pieces of raw attack traffic,[7] as shown in Table 7. The purpose of the experiment is that the AAT created by the attacker on the surrogate model misclassifies the target model.

As shown in Table 8, the target models are on rows, and the surrogate models are on columns. The transferability of the AAT is indicated by the detection rate of the target model against the attacks. The last row is the detection rate of the target model against the raw attack traffic. It is evident that both data augmentation (i.e., SIM, DGM) and advanced optimization algorithms (i.e., MI-FGSM, VMI-FGSM) can generate AAT with high transferability within SPTS. For instance, on the CIC-IDS-2018, the average detection rate of the AAT generated by using VMI-FGSM is only 43.1%, where the surrogate model is MLP. On the other hand, unlike images, the transferability is not necessarily enhanced by a straightforward overlay of attack algorithms. For example, on the CIC-IDS-2018, the average detection rate of the AAT crafted with MI-FGSM is 55.6% which is lower than the 66.8% of SIM, where the surrogate model is LSTM. The same results are presented on the TON_IoT. The average detection rate of the AAT crafted with MI-FGSM is 75.7%, which is lower than the 79.6% of VMI-FGSM, where the surrogate model is CNN. We believe there are several causes behind this: (1) The number of iterations and step size are set uniformly for all algorithms within SPTS; (2) SPTS limits the perturbation dimensions; etc.

Table 6
The raw attack traffic for the CIC-IDS-2018.

Attack	Statistics
DDoS attacks-LOIC-HTTP	28 360
DDOS attack-HOIC	15 840
Bot	13 848
SSH-Bruteforce	9188
DoS attacks-Hulk	8441
Infiltration	5467
DoS attacks-GoldenEye	2615
DoS attacks-Slowloris	695
DDOS attack-LOIC-UDP	163
Brute Force -Web	43
Brute Force -XSS	21
SQL Injection	7
All	84 688

Table 7
The raw attack traffic for the TON_IoT.

Attack	Statistics
XSS	169 886
Password	159 569
DDoS	143 096
Scanning	47 118
Injection	42 427
DoS	10 475
Backdoor	2947
Ransomware	2656
MITM	98
All	578 272

[7] The raw PCAP files are recounted with CICFlowMeter.

Table 8
The detection rate of raw attack traffic and AAT. The lower the detection rate, the higher the transferability of AAT.

Model	Attack	MLP-t (%)	CNN-t (%)	ResCNN-t (%)	LSTM-t (%)	Self-attention-t (%)	Average (%)
		IDS18/TON	IDS18/TON	IDS18/TON	IDS18/TON	IDS18/TON	IDS18/TON
MLP-s	MI-FGSM	62.5/88.5	59.9/89.9	61.7/78.8	62.0/57.4	61.5/89.4	61.5/80.8
	SIM	62.5/88.8	60.0/90.6	61.7/79.6	62.0/63.1	61.5/87.8	61.5/82.0
	VMI-FGSM	59.8/86.7	53.0/86.8	41.1/69.7	44.4/31.6	17.2/87.7	43.1/72.5
	DGM	77.0/85.9	53.7/86.6	45.3/67.4	53.2/25.1	22.3/86.6	50.3/70.3
CNN-s	MI-FGSM	60.0/83.8	53.8/87.5	41.1/74.8	55.9/40.7	25.4/91.6	47.2/75.7
	SIM	60.0/82.5	53.8/85.5	41.1/70.4	56.1/32.5	25.5/90.2	47.3/72.2
	VMI-FGSM	67.7/84.4	54.9/88.1	65.2/78.0	59.6/55.6	35.8/92.1	56.6/79.6
	DGM	60.3/83.8	55.7/86.3	45.5/72.5	50.9/39.5	26.4/91.0	47.8/74.6
ResCNN-s	MI-FGSM	81.3/83.0	73.9/82.9	79.2/75.4	78.0/46.4	72.5/90.0	77.0/75.5
	SIM	81.4/86.1	73.8/88.3	78.7/68.7	77.7/40.9	72.3/90.3	76.8/74.9
	VMI-FGSM	72.9/84.9	59.4/84.6	65.5/72.0	57.6/35.4	49.6/90.4	61.0/73.5
	DGM	69.5/85.5	62.4/86.6	66.5/70.0	62.0/37.2	52.4/88.6	62.6/73.6
LSTM-s	MI-FGSM	60.3/88.2	60.0/90.4	66.6/82.7	51.1/52.3	40.0/92.4	55.6/81.2
	SIM	93.5/88.7	60.2/90.5	60.2/78.3	68.2/37.2	52.0/89.9	66.8/76.9
	VMI-FGSM	70.0/87.2	57.5/89.0	62.5/74.9	48.9/33.3	36.3/92.3	55.0/75.3
	DGM	75.5/87.6	66.4/89.6	71.9/76.8	65.8/45.5	54.9/91.7	66.9/78.2
Self-attention-s	MI-FGSM	93.1/89.2	86.1/92.9	92.8/79.7	93.1/71.0	68.8/90.4	86.8/84.6
	SIM	93.6/89.0	86.9/92.4	75.0/79.2	84.7/68.2	67.9/90.2	81.6/83.8
	VMI-FGSM	89.2/88.6	81.6/91.6	86.7/78.3	81.1/65.5	78.3/92.0	83.4/83.2
	DGM	84.9/88.3	75.4/91.3	81.1/78.9	77.2/66.7	71.8/91.7	78.1/83.4
Raw attack traffic		93.8/99.4	93.9/99.5	93.9/99.1	93.9/99.4	93.8/99.3	93.9/99.3

Another interesting phenomenon is that the average detection rate of NIDSs trained on TON_IoT is generally higher than that of those trained on CIC-IDS-2018. We believe that this may be related to the traffic distribution. The number of attack traffic in TON_IoT is significantly larger than that in CIC-IDS-2018, which is very useful for the model to learn the behavioral patterns of malicious traffic.

DGM exhibits strong performance in all comparison experiments. For example, DGM's detection rate on both datasets is 10 percentage points lower than MI-FGSM's when the surrogate model is MLP. In other experiments, DGM's performance is suboptimal, if not optimal. In conclusion, the experimental results in Table 8 demonstrate the effectiveness of SPTS and DGM.

5.3.2. Practical packets

In this section, the AAT is mapped to practical packets and compared with the existing method, TANTRA (Sharon et al., 2022). The complete experimental procedure for SPTS is as follows: (1) Apply CICFlowMeter to extract features from the raw attack traffic, which is stored in PCAP files; (2) Use the extracted features and SPTS to generate AAT based on five surrogate models; (3) Modify the packets in the PCAP files based on the generated AAT; (4) Re-extract features from the modified packets with CICFlowMeter; (5) Input the resulting features into the target NIDSs for evaluation. For TANTRA, the complete experimental procedure is as follows: (1) Train an LSTM model according to the configuration described in Sharon et al. (2022), with a window size set to 150; (2) Use the trained LSTM to modify the $Timestamp$ of each packet; (3) Re-extract features from the modified packets with CICFlowMeter; (4) Feed the resulting features into the target NIDSs for evaluation. Due to the challenge of mapping partial labels to raw packets in the CIC-IDS-2018, the experiments in this section are conducted only on the TON_IoT. To facilitate modification of the packets, we filter the raw attack traffic and retain only those that are not split into multiple records by CICFlowMeter, resulting in a total of 36,980. Table 9 presents the results of all experiments, averaged over three random runs. The last row of the table indicates the detection rate of the target NIDSs on the raw attack traffic.

In terms of average detection rate, SPTS performs slightly below TANTRA. However, when examining individual experiments, SPTS can generate stronger AAT. For example, when the surrogate model is an LSTM and the attack is SIM, the detection rates of the target NIDSs fall to as low as 82.2%. Beyond this, SPTS offers two significant advantages over TANTRA. First, SPTS requires modifying at most six packets, whereas TANTRA changes the Timestamp of each packet. Second, training the LSTM for TANTRA is non-trivial and demands substantial expertise. Compared to the results in Table 8, the performance of SPTS shows a noticeable decline. We attribute this to two primary factors. First, statistical discrepancies are introduced during the process of mapping AAT back to actual packets and re-extracting features from them. Second, some features, such as $Pkt\ Len\ Mean$, are theoretically calculated based on the equations presented in Table 2. However, the actual statistics deviate significantly from the expected values. These statistical deviations will be addressed in our future work. Despite these errors, SPTS can still reduce the detection rates of the target NIDSs, demonstrating the effectiveness of our approach.

5.4. Ablation study

5.4.1. SPTS

There are two hyperparameters in SPTS, namely, the number of iterations T and step size l. We performed ablation experiments on them. Figs. 2 and 3 exhibit detection rates with different T and l evaluated on the CIC-IDS-2018 and TON_IoT, where the lighter the color, the lower the detection rate (the higher the transferability). We generate AAT with DGM and MLP-s and then evaluate the detection rate on the five target models for both datasets. The choice of T and l is comparable to choosing the appropriate epoch and learning rate when training DNNs. Poor combinations of T and l produce poor transferability, such as 3 and 100 (with a 93.7% detection rate in Fig. 2a) or 5 and 80 (with a 90.6% detection rate in Fig. 3b). Better combinations of T and l produce higher transferability, such as 7 and 160 (with a 42.1% detection rate in Fig. 2c) or 13 and 180 (with a 21.2% detection rate in Fig. 3d).

Intuitively, the more iterations and the more significant the step size, the more transferable AAT is. While Fig. 2 is not prominent, Fig. 3 is very obvious. We attempt to explain this phenomenon based on the perspective of traffic distribution. The CIC-IDS-2018 contains much normal and relatively little malicious traffic. The TON_IoT, on the contrary, has more malicious traffic. We argue that it is easier for

Table 9
The detection rate of the AAT extracted from practical packets. The lower the detection rate, the higher the transferability of AAT.

Model	Attack	MLP-t (%)	CNN-t (%)	ResCNN-t (%)	LSTM-t (%)	Self-attention-t (%)	Average (%)
MLP-s	MI-FGSM	91.5	95.5	91.3	96.3	96.7	94.3
	SIM	96.4	95.5	90.7	96.2	96.8	95.1
	VMI-FGSM	82.5	90.0	84.4	96.0	96.4	89.9
	DGM	82.9	94.3	83.3	96.0	96.4	90.6
CNN-s	MI-FGSM	93.3	95.9	92.9	96.7	97.2	95.2
	SIM	90.5	94.9	84.1	96.4	96.3	92.4
	VMI-FGSM	93.0	91.2	92.0	96.6	96.7	93.9
	DGM	88.3	93.2	86.5	96.3	96.4	92.2
ResCNN-s	MI-FGSM	92.9	92.2	88.9	96.4	96.7	93.4
	SIM	83.6	95.6	86.4	96.6	97.0	91.8
	VMI-FGSM	85.0	88.8	90.2	96.3	96.4	91.3
	DGM	83.5	94.1	85.6	96.4	96.8	91.3
LSTM-s	MI-FGSM	89.1	95.5	90.4	96.1	96.9	93.6
	SIM	82.2	95.8	92.6	96.2	96.8	92.7
	VMI-FGSM	83.3	91.5	86.5	95.8	96.3	90.7
	DGM	87.6	90.5	89.7	96.2	96.5	92.1
Self-attention-s	MI-FGSM	87.8	96.1	92.7	96.9	97.0	94.1
	SIM	83.8	96.1	95.2	97.0	97.1	93.8
	VMI-FGSM	90.0	92.4	92.4	97.0	97.0	93.8
	DGM	87.8	92.7	93.0	96.8	96.8	93.4
Tantra		90.6	91.7	92.2	84.7	86.9	89.2
Raw attack traffic		98.2	97.9	97.3	98.2	97.9	97.9

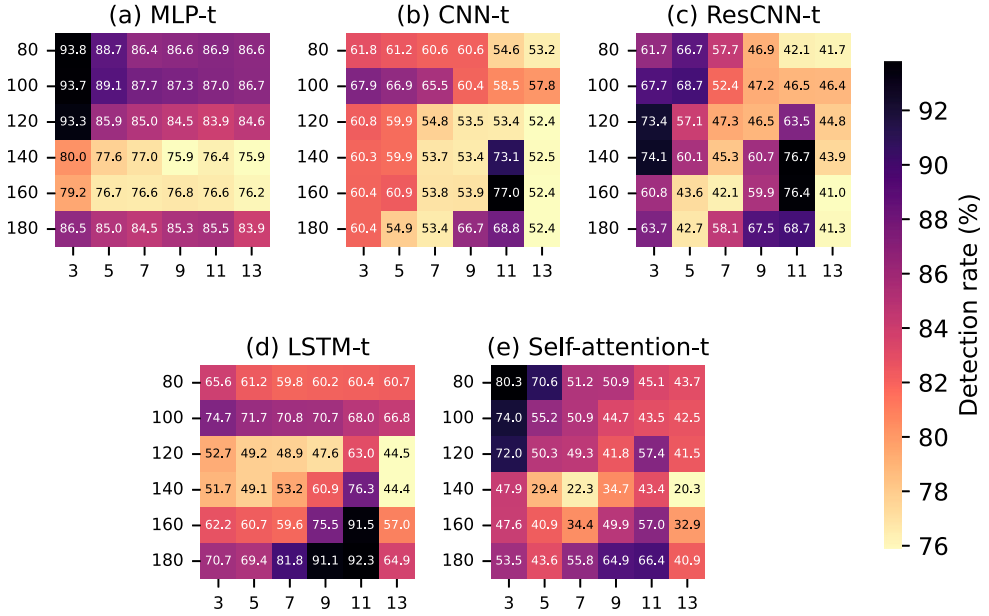


Fig. 2. Effect of various T and l on transferability in SPTS evaluated on the CIC-IDS-2018. In the subfigure, each column represents a different T and each row represents a different l. The names of the subfigures indicate target models.

surrogate models trained on TON_IoT to learn the characteristics of malicious traffic. Therefore, the transferability of AAT generated on these models is more regular.

5.4.2. DGM

There are two hyperparameters in DGM that need to be predefined: the number of copies and the dropout rate. We performed ablation experiments for each of them. Figs. 4 and 5 show the experimental results on the CIC-IDS-2018 and TON_IoT, respectively. The dropout rate is fixed at 0.2, and the number of copies varied in these experiments. As the number of copies changes, the transferability of the generated AAT also changes. Intuitively, the more copies there are, the more robust the generated gradient is, and the higher the transferability of AAT will be. However, this is not always the case. As shown in Figs. 4d, e, 5d, and e the transferability of AAT decreases as the number of copies increases. We note that the input data for both sets of experiments are transformed into sequence data, while the transformation of DGM is in the spatial domain. This may be an important reason for this phenomenon, which we will investigate further in future work. Figs. 6 and 7 illustrate the performance of the DGM for different dropout rates while keeping the number of copies fixed at 5. Changes in the dropout rate do not result in a noticeable effect on transferability in

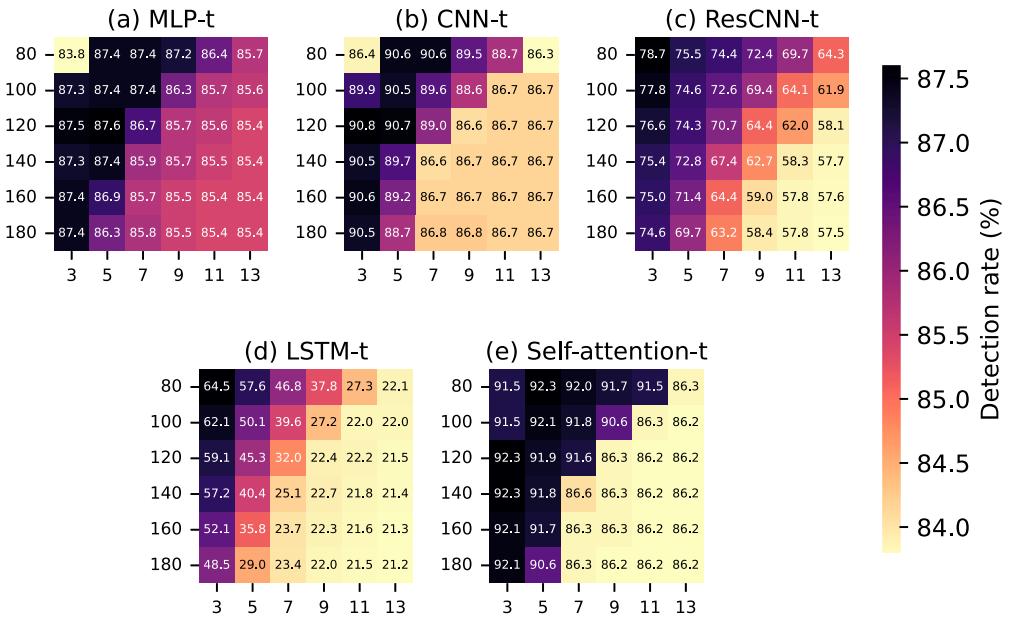


Fig. 3. Effect of various T and l on transferability in SPTS evaluated on the TON_IoT. In the subfigure, each column represents a different T and each row represents a different l. The names of the subfigures indicate target models.

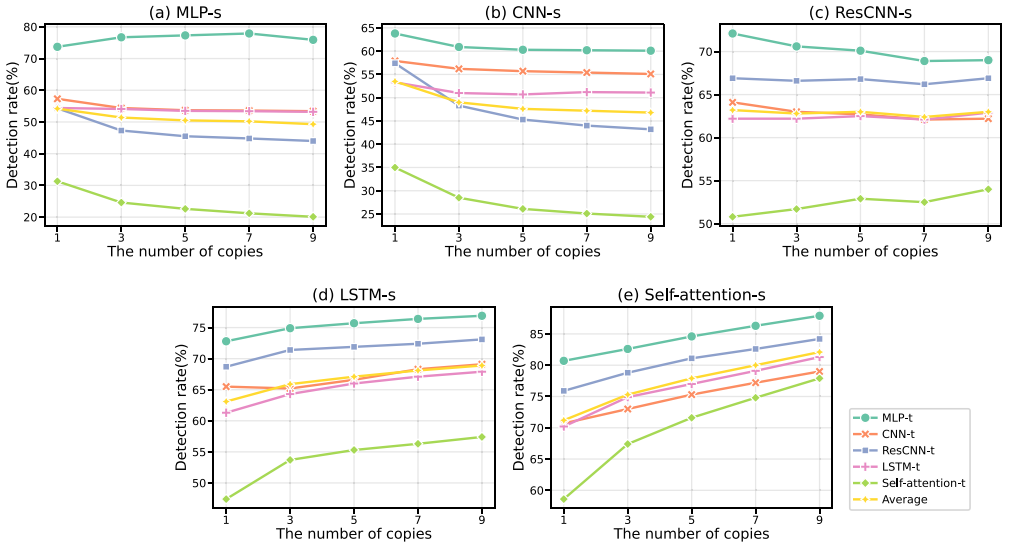


Fig. 4. Impact of the number of copies on transferability in DGM evaluated on the CIC-IDS-2018. The names of the subfigures indicate surrogate models.

all experiments. We believe it is due to the random nature of DGM, i.e., the transformations used by DGM are stochastic. Overall, the DGM performs stably with a dropout rate of 0.2, so the dropout rate p is set to 0.2.

5.4.3. Dataset

To further investigate the impact of the dataset on attack, we split the test and training datasets for both the target NIDSs and surrogate models into four different ratios: 1:9, 2:8, 3:7, and 4:6, respectively. Table 10 presents the accuracy, precision, recall, and F1-score of the trained models. In this set of experiments, the surrogate model is an MLP, and the attack algorithms include MI-FGSM, SIM, VMI-FGSM, and DIM. The AAT is generated on 4 MLPs and evaluated on 20 target NIDSs. The detailed results are shown in Tables 11, 12, 13, and 14, where split ratios of target NIDSs are 1:9, 2:8, 3:7, and 4:6, respectively. The first column of each table indicates split ratios of the surrogate models, and the detection rate is used to measure AAT's evasion. The attack traffic in the training dataset for surrogate models is considered the raw attack traffic.

Intuitively, when the target NIDSs are trained on larger training datasets while the surrogate models are trained on smaller ones, the generated AAT would be easier to detect, as more training data may improve the generalization ability of the target NIDSs. However, this trend is not always observed in practice. We believe this is primarily due to the strength of the attack algorithms, which provide robust gradients for creating transferable adversarial examples. Notably, when the split ratio is 3:7, the target NIDSs trained on the TON_IoT achieve exceptionally strong performance in terms of average detection rate. This could be attributed to randomness in the dataset splitting process. Overall, the experimental results demonstrate that the AAT generated by SPTS and DGM exhibits high transferability across different dataset split ratios.

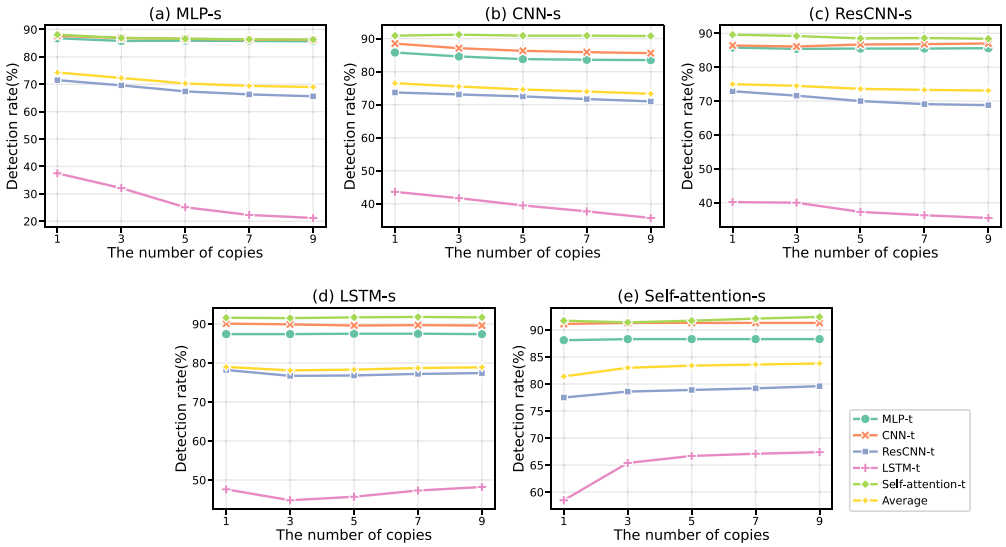


Fig. 5. Impact of the number of copies on transferability in DGM evaluated on the TON_IoT. The names of the subfigures indicate surrogate models.

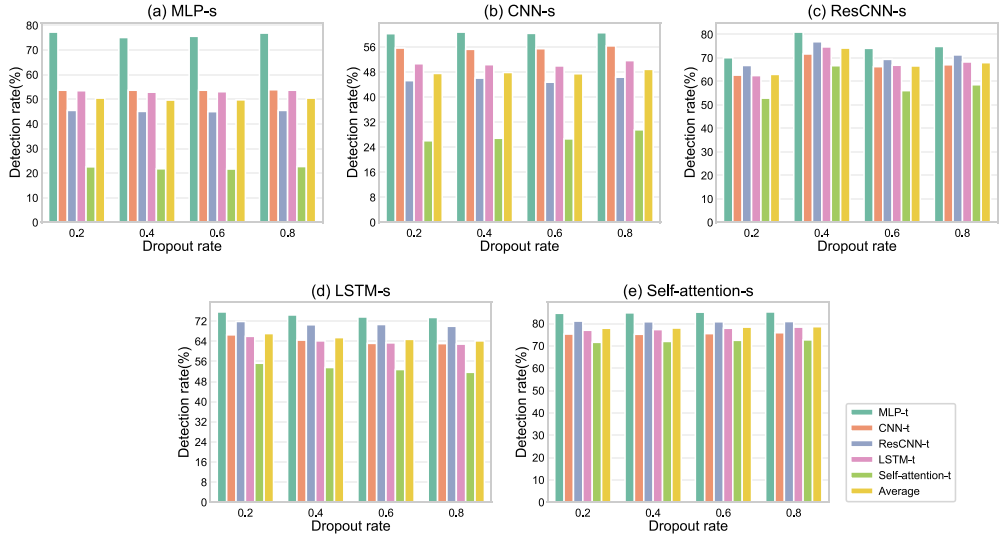


Fig. 6. Impact of dropout rates on transferability in DGM evaluated on the CIC-IDS-2018. The names of the subfigures indicate surrogate models.

5.4.4. *Input features of surrogate models*

As noted in Section 3, many traffic collection and analysis tools are publicly available, allowing attackers to obtain partial knowledge of the features utilized by target NIDSs. In previous experiments, we simulate this scenario by randomly removing six features, as illustrated in Table 4. This section examines the impact of removing various features on the attack. It is essential to note that, to ensure that the generated AAT is practical, all level 1, level 2, and level 3 features are crucial for training the surrogate model. In addition, the following level 0 features are also required: *Tot Fwd Pkts*, *Tot Bwd Pkts*, *TotLen Bwd Pkts*, *Bwd Pkt Len Max*, and *Bwd Pkt Len Min*, as the feature rectifications in Table 2 depend on them. This implies that we assume the attacker knows the features mentioned above, while the number of remaining features is treated as a variable in our experiments.

A total of 39 removable features have been identified. Accordingly, the number of input features used by the surrogate model ranges from 27 to 66. We configured 14 different surrogate models by varying the number in steps of 3 across this range. For each setting, the features to be removed are selected at random. The surrogate model used is an MLP, following the same architecture described in Section 5.1.2, except that the number of units in the input layer is adjusted according to the number of input features. After training the surrogate model, AAT is generated with MI-FGSM, SIM, VMI-FGSM, and DGM and tested on different target NIDSs. Each experiment is conducted three times, and the average results are calculated. The experimental results are presented in Figs. 8 and 9, corresponding to the CIC-IDS-2018 and the TON_IoT, respectively.

The results shown in Fig. 8 regarding the CIC-IDS-2018 indicate that the input features utilized by the surrogate model can affect attacks. For example, when the target NIDS is an MLP, the detection rate reaches 93.8%. As the number of input features increases, the detection rate decreases. However, we believe this trend is primarily due to the relatively low proportion of attack traffic in CIC-IDS-2018. This hypothesis is supported by the results illustrated in Fig. 9 on the TON_IoT, where the detection rates of the target NIDSs remain relatively stable regardless of the number of input features. These findings suggest that while the input features of the surrogate model can affect attack effectiveness, the degree of that effect is mainly dictated by the underlying data distribution.

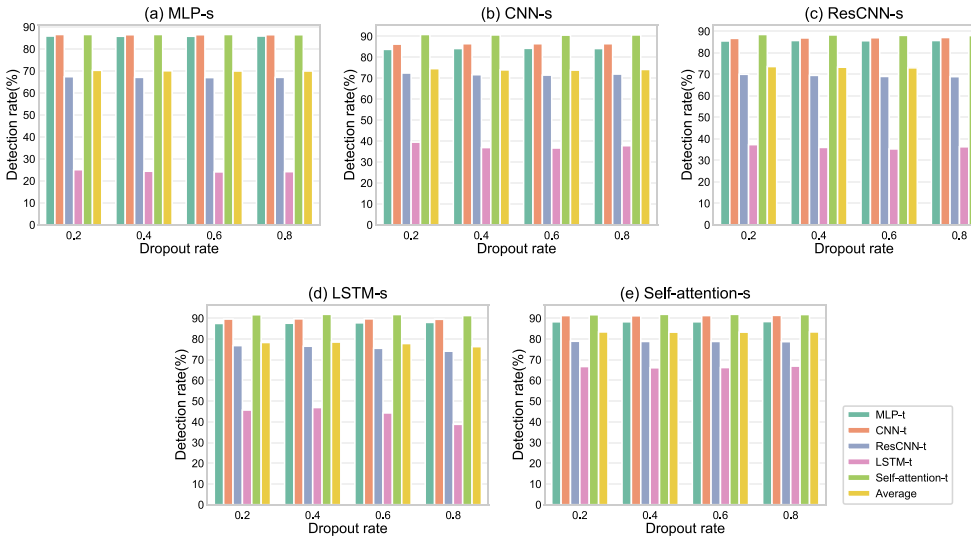


Fig. 7. Impact of dropout rates on transferability in DGM evaluated on the TON_IoT. The names of the subfigures indicate surrogate models.

Table 10
The detection performance of models with different dataset split ratios.

Model	Ratio	Accuracy (%)	Precision (%)	Recall (%)	F1 score (%)
		IDS18/TON	IDS18/TON	IDS18/TON	IDS18/TON
MLP-t	1:9	98.7/98.9	97.5/99.9	94.9/98.9	96.2/99.4
	2:8	98.7/98.6	97.1/99.9	94.8/98.6	96.0/99.3
	3:7	98.6/98.8	96.4/99.9	94.8/98.7	95.6/99.4
	4:6	98.7/98.9	97.2/99.9	94.9/98.9	96.0/99.4
CNN-t	1:9	98.7/98.8	96.8/99.9	95.4/98.8	96.1/99.4
	2:8	98.6/98.4	96.7/99.9	94.9/98.4	95.8/99.2
	3:7	98.7/99.0	97.3/99.9	94.9/99.0	96.1/99.5
	4:6	98.7/98.9	96.9/99.9	95.4/98.9	96.1/99.4
ResCNN-t	1:9	98.7/99.0	97.5/99.9	94.8/99.0	96.1/99.5
	2:8	98.6/98.2	96.7/99.9	94.8/98.2	95.7/99.1
	3:7	98.7/98.9	96.9/99.9	95.3/98.9	96.1/99.4
	4:6	98.7/99.0	97.4/99.9	94.5/99.0	96.0/99.5
LSTM-t	1:9	98.8/99.0	97.6/99.9	94.9/99.0	96.2/99.5
	2:8	98.7/98.7	97.3/99.9	94.9/98.7	96.1/99.3
	3:7	98.8/98.9	97.4/99.9	95.4/98.9	96.4/99.4
	4:6	98.8/99.0	97.4/99.9	95.3/99.0	96.4/99.5
Self-attention-t	1:9	98.7/98.4	97.2/99.9	94.9/98.4	96.0/99.2
	2:8	98.6/98.2	96.7/99.9	94.9/98.3	95.7/99.1
	3:7	98.6/98.6	96.4/99.9	94.9/98.6	95.6/99.3
	4:6	98.7/97.9	97.0/99.9	94.9/97.9	95.9/98.9
MLP-s	1:9	98.3/98.7	95.1/99.9	94.8/98.7	95.0/99.3
	2:8	98.7/98.6	97.2/99.9	94.9/98.6	96.0/99.3
	3:7	98.9/98.8	99.9/99.9	93.3/98.7	96.5/99.3
	4:6	98.7/98.8	97.5/99.9	94.9/98.8	96.2/99.4

5.5. Execution cost

A necessary condition for the successful launching of an attack is an acceptable execution cost, which we will analyze in terms of the time cost, the length increment of AAT, and the delay increment of AAT, respectively. It is easy to see from Algorithm 1 that the time complexity of STPS is related to the algorithm's logic and the surrogate model. Regarding the algorithm's logic, three key factors increase the time overhead: iteration, gradient calculation, and feature rectification. Algorithm 1 demonstrates that if the gradient depends only on the momentum and the iterations are T, the time complexity is $O(T)$, e.g., MI-FGSM. If the gradient relies not only on the momentum but also on the copies and the number of copies is N, the time complexity is $O(T \times N)$, e.g., SIM, VMI-FGSM, and DGM. As shown in Tables 15 and 16, the average time to generate one AAT for MI-FGSM is less than the other methods when the surrogate models are the same. Feature rectification also increases the time overhead of the attack because it corrects traffic features level by level. However, no matter which surrogate model is chosen, the average time to generate one AAT is no more than 4000 μs. Obviously, the time cost of feature rectification is acceptable to the attacker. In addition, the choice of surrogate models also significantly impacts the time cost of the attack. We believe that this is because different models have different numbers of parameters.

Table 11
The detection rate of raw attack traffic and AAT. The dataset split ratio for the target NIDSs was set to 1:9.

Ratio	Attack	MLP-t (%)	CNN-t (%)	ResCNN-t (%)	LSTM-t (%)	Self-attention-t (%)	Average (%)
		IDS18/TON	IDS18/TON	IDS18/TON	IDS18/TON	IDS18/TON	IDS18/TON
1:9	MI-FGSM	44.1/98.2	59.7/91.8	59.9/83.0	44.4/62.2	44.0/94.5	50.4/86.0
	SIM	77.1/98.7	85.7/93.6	81.0/81.3	77.8/72.5	77.1/97.5	79.8/88.7
	VMI-FGSM	43.9/98.2	52.2/87.4	49.4/76.4	44.4/57.7	43.7/92.9	46.7/82.5
	DGM	43.9/98.2	53.6/88.1	49.1/73.9	44.6/56.1	43.7/93.0	47.0/81.9
	Raw attack traffic	93.8/99.5	93.9/99.6	93.8/99.5	93.7/99.6	93.8/98.9	93.8/99.4
2:8	MI-FGSM	61.5/98.7	62.5/92.8	61.4/82.7	51.6/69.1	59.8/94.0	59.4/87.4
	SIM	61.5/98.4	62.6/93.1	62.6/89.0	63.0/71.2	59.8/96.7	61.9/89.7
	VMI-FGSM	43.9/98.4	52.6/87.4	49.8/75.8	33.0/57.5	43.8/92.8	44.6/82.4
	DGM	47.5/98.2	71.7/88.5	66.9/76.4	50.6/58.2	43.8/93.2	56.1/82.9
	Raw attack traffic	93.8/99.5	93.9/99.6	93.8/99.5	93.7/99.6	93.8/98.9	93.8/99.4
3:7	MI-FGSM	77.7/98.5	86.8/92.2	82.5/79.9	77.7/62.0	77.7/93.6	80.5/85.2
	SIM	77.7/98.3	86.8/89.2	82.6/78.4	77.7/66.8	77.8/96.0	80.5/85.7
	VMI-FGSM	43.9/98.3	52.5/87.4	48.5/75.7	44.5/56.9	43.8/92.8	46.6/82.2
	DGM	44.9/98.2	54.8/88.0	50.4/73.8	45.9/55.9	44.3/92.8	48.1/81.8
	Raw attack traffic	93.6/99.5	93.8/99.6	93.7/99.5	93.6/99.6	93.6/98.9	93.7/99.4
4:6	MI-FGSM	77.4/98.3	85.8/89.5	84.8/75.6	77.8/57.7	77.4/94.1	80.7/83.0
	SIM	93.6/98.2	93.4/92.6	89.9/84.3	93.5/78.1	93.6/96.8	92.8/90.0
	VMI-FGSM	44.1/98.3	52.7/88.2	49.5/74.6	44.6/56.5	43.8/92.9	46.9/82.1
	DGM	53.8/98.3	68.6/88.0	63.4/74.3	56.3/56.5	49.6/93.0	58.3/82.0
	Raw attack traffic	93.8/99.5	93.9/99.6	93.9/99.5	93.7/99.6	93.8/98.9	93.8/99.4

Table 12
The detection rate of raw attack traffic and AAT. The dataset split ratio for the target NIDSs was set to 2:8.

Ratio	Attack	MLP-t (%)	CNN-t (%)	ResCNN-t (%)	LSTM-t (%)	Self-attention-t (%)	Average (%)
		IDS18/TON	IDS18/TON	IDS18/TON	IDS18/TON	IDS18/TON	IDS18/TON
1:9	MI-FGSM	60.1/85.4	60.0/87.3	60.0/78.1	60.2/55.4	44.1/92.6	56.9/79.8
	SIM	93.2/86.4	85.9/90.4	74.3/79.5	83.7/43.0	50.5/92.2	77.5/78.3
	VMI-FGSM	59.8/86.6	52.4/87.0	40.8/69.3	60.1/28.9	17.0/86.5	46.0/71.7
	DGM	59.8/85.7	52.7/86.1	41.8/65.7	50.1/20.9	18.3/87.1	44.6/69.1
	Raw attack traffic	93.8/99.4	93.9/99.5	93.9/99.1	93.9/99.4	93.8/99.3	93.9/99.3
2:8	MI-FGSM	62.5/88.5	59.9/89.9	61.7/78.8	62.0/57.4	61.5/89.4	61.5/80.8
	SIM	62.5/88.8	60.0/90.6	61.7/79.6	62.0/63.1	61.5/87.8	61.5/82.0
	VMI-FGSM	59.8/86.7	53.0/86.8	41.1/69.7	44.4/31.6	17.2/87.7	43.1/72.5
	DGM	77.0/85.9	53.7/86.6	45.3/67.4	53.2/25.1	22.3/86.6	50.3/70.3
	Raw attack traffic	93.8/99.4	93.9/99.5	93.9/99.1	93.9/99.4	93.8/99.3	93.9/99.3
3:7	MI-FGSM	93.6/86.1	86.8/88.0	93.7/76.4	93.8/45.1	70.8/87.8	87.7/76.7
	SIM	93.6/85.8	86.8/86.9	93.7/66.2	93.8/31.2	70.8/86.5	87.7/71.3
	VMI-FGSM	60.0/85.8	52.9/86.8	41.2/69.0	60.4/25.3	17.2/87.5	46.4/70.9
	DGM	61.0/85.7	53.4/86.0	44.4/65.2	51.5/20.9	19.7/86.9	46.0/68.9
	Raw attack traffic	93.7/99.4	93.8/99.5	93.8/99.1	93.8/99.4	93.7/99.3	93.7/99.3
4:6	MI-FGSM	93.1/85.9	86.0/87.7	93.1/67.8	93.7/28.5	70.3/86.4	87.2/71.3
	SIM	93.6/88.7	93.6/87.2	93.6/73.3	93.7/51.1	93.6/92.5	93.6/78.6
	VMI-FGSM	60.1/85.7	52.8/86.8	41.5/65.4	60.3/21.5	17.2/86.3	46.4/69.1
	DGM	70.6/85.7	60.8/86.1	55.9/65.8	55.1/21.8	30.4/86.9	54.6/69.3
	Raw attack traffic	93.9/99.4	93.9/99.5	93.9/99.1	93.9/99.4	93.9/99.3	93.9/99.3

SPTS increases the maximum packet length and decreases the minimum packet length (refer to Section 4). Intuitively, decreasing the packet length decreases the attack cost, so only the increment of maximum packet length is considered in this section. As indicated in Tables 15 and 16, the average increase of maximum packet length is no more than 900 bytes. This is a very lightweight overhead for the attacker since it only needs to be increased for one packet per traffic, not all packets. Finally, we count the delay increment by the different attack methods. As mentioned in Section 4, the increment of maximum delay does not exceed 980 μs due to the constraints from the two hyperparameters, namely iteration and step size, which generally do not lead to the termination of the connection. Overall, the cost of launching the attack is very low.

5.6. Mitigating adversarial attacks

We use adversarial training as a defense mechanism to further evaluate the effectiveness of the proposed SPTS and DGM and defend against such attacks. Adversarial training is a strategy to improve model robustness by actively introducing adversarial examples to enhance the model's defense against malicious perturbation. In CV, adversarial training is the most effective defense method (He et al., 2023), so this method is also chosen in our work. As shown in Tables 17 and 18, we conduct two forms of adversarial training: normal adversarial training and adversarial training with SPTS.

For normal adversarial training, the traffic is treated as unstructured data during training. Adversary examples are generated for malicious traffic in each batch, and the attack used is MI-FGSM. As shown in Table 17, the adversary-trained NIDSs perform more robustly than the normal-trained NIDSs (refer to Table 8). The conclusion is that

Table 13
The detection rate of raw attack traffic and AAT. The dataset split ratio for the target NIDSs was set to 3:7.

Ratio	Attack	MLP-t (%)	CNN-t (%)	ResCNN-t (%)	LSTM-t (%)	Self-attention-t (%)	Average (%)
		IDS18/TON	IDS18/TON	IDS18/TON	IDS18/TON	IDS18/TON	IDS18/TON
1:9	MI-FGSM	60.1/95.9	60.0/93.7	59.7/95.0	59.9/99.0	44.1/90.5	56.8/94.8
	SIM	93.5/97.3	93.1/98.2	92.8/98.1	93.3/99.0	77.4/89.4	90.0/96.4
	VMI-FGSM	60.1/95.5	59.6/96.4	59.3/97.7	59.8/98.9	43.9/87.8	56.5/95.2
	DGM	60.0/95.6	59.6/96.2	59.4/97.5	59.9/98.9	43.9/87.7	56.6/95.2
	Raw attack traffic	93.8/99.5	93.9/99.6	93.8/99.6	93.9/99.6	93.7/99.4	93.8/99.5
2:8	MI-FGSM	60.2/96.6	59.8/97.5	59.8/97.9	57.7/99.0	59.8/91.0	59.4/96.4
	SIM	60.2/96.7	59.9/97.3	59.8/97.9	62.9/99.0	59.8/90.3	60.5/96.2
	VMI-FGSM	60.1/95.6	59.8/96.1	59.8/97.4	54.8/98.9	44.0/87.7	55.7/95.2
	DGM	60.1/95.4	60.2/96.1	60.3/97.5	72.5/98.9	44.1/87.7	59.4/95.1
	Raw attack traffic	93.8/99.5	93.9/99.6	93.8/99.6	93.9/99.6	93.7/99.4	93.8/99.5
3:7	MI-FGSM	93.6/96.0	93.7/97.3	93.7/97.9	93.5/99.0	77.7/89.2	90.4/95.9
	SIM	93.6/95.7	93.7/96.9	93.7/97.7	93.5/98.9	77.7/87.7	90.4/95.4
	VMI-FGSM	60.3/95.5	59.8/96.3	59.5/97.5	60.1/98.9	43.9/87.7	56.7/95.2
	DGM	60.5/95.4	60.0/96.2	59.8/97.4	61.4/98.9	44.3/87.7	57.2/95.1
	Raw attack traffic	93.6/99.5	93.8/99.7	93.7/99.6	93.7/99.6	93.5/99.4	93.7/99.5
4:6	MI-FGSM	93.6/96.2	93.2/97.4	92.9/97.9	93.4/98.9	77.5/87.8	90.1/95.6
	SIM	93.6/97.0	93.7/96.8	93.4/97.2	93.4/99.0	93.5/91.2	93.5/96.3
	VMI-FGSM	60.5/95.4	60.0/96.2	59.7/97.4	60.3/98.9	44.1/87.7	56.9/95.1
	DGM	65.8/95.4	65.4/96.2	65.1/97.4	73.4/98.9	49.7/87.7	63.9/95.1
	Raw attack traffic	93.8/99.5	93.9/99.6	93.9/99.6	93.9/99.6	93.7/99.4	93.8/99.5

Table 14
The detection rate of raw attack traffic and AAT. The dataset split ratio for the target NIDSs was set to 4:6.

Ratio	Attack	MLP-t (%)	CNN-t (%)	ResCNN-t (%)	LSTM-t (%)	Self-attention-t (%)	Average (%)
		IDS18/TON	IDS18/TON	IDS18/TON	IDS18/TON	IDS18/TON	IDS18/TON
1:9	MI-FGSM	59.9/91.1	60.1/78.0	44.1/85.8	53.8/83.8	44.0/87.4	52.4/85.2
	SIM	74.3/89.1	86.1/76.3	70.2/82.7	72.3/93.3	70.1/94.1	74.6/87.1
	VMI-FGSM	40.8/89.4	52.6/71.0	36.8/80.9	39.0/82.4	36.7/85.2	41.2/81.8
	DGM	40.9/88.6	53.9/70.4	36.8/80.4	38.7/82.3	37.0/84.1	41.5/81.1
	Raw attack traffic	93.8/99.5	93.9/99.6	93.8/99.7	93.8/99.5	93.8/99.1	93.8/99.5
2:8	MI-FGSM	61.5/90.6	59.9/74.8	59.7/85.2	48.9/90.6	48.3/93.2	55.7/86.9
	SIM	61.6/91.0	59.9/78.4	59.8/88.3	60.1/84.3	59.7/88.6	60.2/86.1
	VMI-FGSM	41.0/89.5	53.1/70.5	36.6/80.5	28.2/82.6	25.4/84.6	36.8/81.5
	DGM	44.7/88.7	54.4/70.6	36.8/80.8	28.4/83.0	25.9/84.5	38.1/81.5
	Raw attack traffic	93.8/99.5	93.9/99.6	93.8/99.7	93.8/99.6	93.8/99.0	93.8/99.5
3:7	MI-FGSM	93.6/88.6	86.8/72.1	70.8/82.5	91.5/84.9	70.8/88.9	82.7/83.4
	SIM	93.6/88.5	86.8/71.5	70.8/81.2	91.5/87.7	70.8/89.3	82.7/83.7
	VMI-FGSM	41.0/88.5	53.0/70.5	36.7/80.5	39.2/82.2	36.9/84.7	41.3/81.3
	DGM	42.2/88.5	54.2/70.1	37.7/79.9	39.8/82.2	37.8/83.8	42.3/80.9
	Raw attack traffic	93.7/99.5	93.8/99.6	93.7/99.7	93.6/99.6	93.6/99.0	93.7/99.5
4:6	MI-FGSM	93.1/88.6	86.2/72.3	70.4/81.6	92.8/82.2	70.3/84.9	82.6/81.9
	SIM	93.6/91.2	93.8/82.0	93.7/87.2	91.9/92.1	93.6/91.5	93.3/88.8
	VMI-FGSM	41.1/88.5	53.1/70.7	37.0/80.3	39.5/82.1	36.9/84.2	41.5/81.2
	DGM	50.3/88.5	62.2/70.4	44.2/80.1	44.4/82.6	45.2/84.1	49.3/81.1
	Raw attack traffic	93.8/99.5	93.9/99.6	93.9/99.7	93.8/99.5	93.8/99.0	93.8/99.5

Table 15
Execution cost of MI-FGSM and SIM within the SPTS framework. "Time cost" refers to the average time to generate one AAT. "Length" refers to the average increase in maximum packet length for one AAT. "Delay" refers to the average increase in maximum delay for one AAT.

Model	MI-FGSM			SIM		
	Time cost (μs)	Length (byte)	Delay (μs)	Time cost (μs)	Length (byte)	Delay (μs)
	IDS18/TON	IDS18/TON	IDS18/TON	IDS18/TON	IDS18/TON	IDS18/TON
MLP-s	1268.1/1292.0	448.8/404.2	667.9/398.2	1813.2/1888.3	326.2/139.3	340.9/495.9
CNN-s	1527.6/1366.6	739.0/487.8	550.1/240.1	3151.8/2843.2	667.6/368.3	742.0/147.5
ResCNN-s	1459.1/1372.7	599.6/190.5	389.9/156.4	3043.1/2847.3	10.2/84.6	2.7/867.7
LSTM-s	1410.6/1454.2	699.7/212.2	254.0/347.2	2851.3/3027.8	297.0/112.9	623.1/418.4
Self-attention-s	1258.0/1451.3	269.1/342.9	276.0/771.6	2206.1/2411.1	297.1/39.2	408.5/883.9

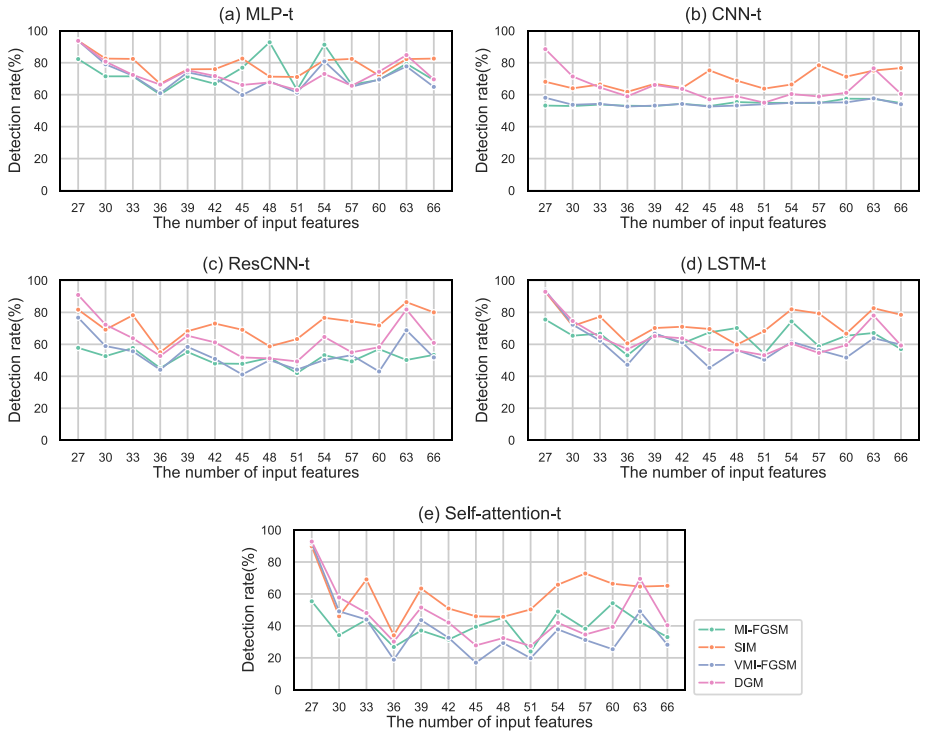


Fig. 8. The impact of input features of surrogate models on attacks for the CIC-IDS-2018. The names of the subfigures indicate target NIDSs.

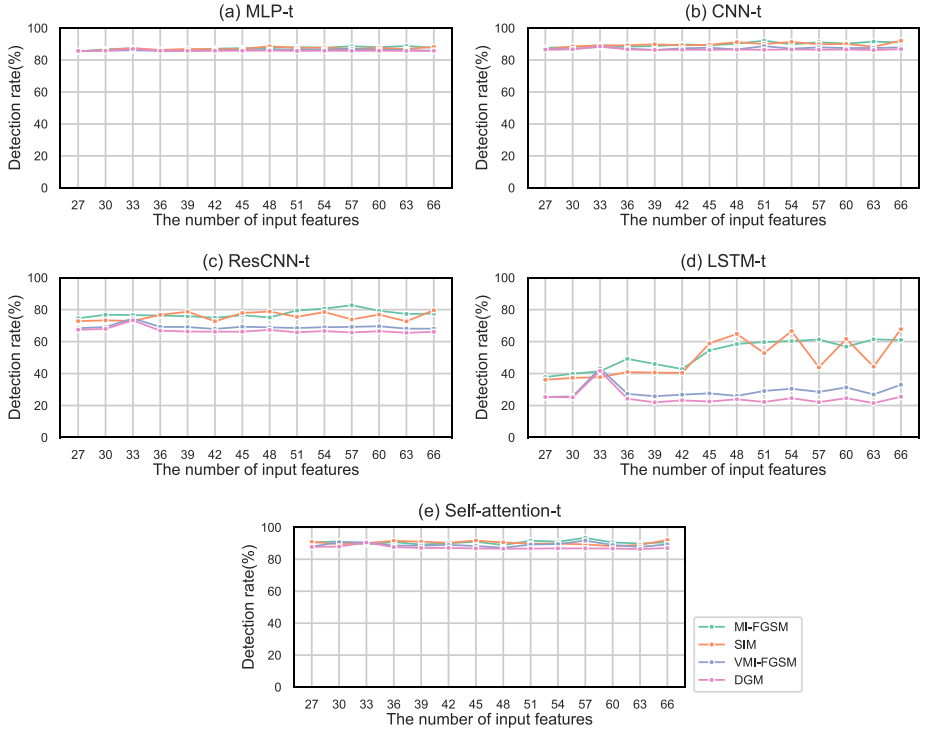


Fig. 9. The impact of input features of surrogate models on attacks for the TON_IoT. The names of the subfigures indicate target NIDSs.

even normal adversarial training that treats traffic as unstructured data can mitigate practical AAT. This may sound counterintuitive, but it is not. The adversarial examples are extremely close to the original data points. If adversarial examples can be taken from the space around the original data points and used for training, the model will be able to learn the characteristics around the data points and thus make judgments. Table 17 also shows that the overall results of the models trained on TON_IoT are performing better than those trained on CIC-IDS-2018, which is consistent with the experimental results in the previous subsections. We believe this is because the TON_IoT contains more samples of malicious traffic, which allows the model to learn appropriately. This also illustrates that if there are enough samples of malicious traffic in the dataset, the defense can be achieved with normal adversarial training.

Table 16
Execution cost of VMI-FGSM and DGM within the SPTS framework. "Time cost" refers to the average time to generate one AAT. "Length" refers to the average increase in maximum packet length for one AAT. "Delay" refers to the average increase in maximum delay for one AAT.

Model	VMI-FGSM			DGM		
	Time cost (μs)	Length (byte)	Delay (μs)	Time cost (μs)	Length (byte)	Delay (μs)
	IDS18/TON	IDS18/TON	IDS18/TON	IDS18/TON	IDS18/TON	IDS18/TON
MLP-s	2031.6/2012.3	876.6/714.5	115.3/258.0	2280.5/2116.7	801.9/744.9	182.2/255.6
CNN-s	3659.8/3398.0	606.8/244.8	263.6/246.1	3856.4/3572.4	714.5/358.2	432.8/365.5
ResCNN-s	3759.0/3450.2	487.4/215.1	814.5/770.1	3999.9/3775.8	441.1/193.1	675.7/565.2
LSTM-s	3396.3/3669.5	539.3/289.1	407.0/433.9	3541.0/3970.6	413.3/274.0	460.6/447.5
Self-attention-s	2511.7/2847.3	174.1/140.1	520.0/539.0	2772.9/3154.4	222.8/121.6	592.8/516.3

Table 17
Normal adversarial training against AAT. The attack in adversarial training is MI-FGSM, and the surrogate model is MLP-s.

Attack	MLP-t (%)	CNN-t (%)	ResCNN-t (%)	LSTM-t (%)	Self-attention-t (%)	Average (%)
	IDS18/TON	IDS18/TON	IDS18/TON	IDS18/TON	IDS18/TON	IDS18/TON
MI-FGSM	84.2/99.9	59.4/91.2	61.3/97.3	82.8/99.9	94.8/99.2	76.5/97.5
SIM	84.2/99.9	59.5/91.2	61.2/97.1	93.7/99.9	94.8/99.2	78.7/97.5
VMI-FGSM	86.0/99.9	59.9/91.1	25.7/97.2	83.0/99.9	94.8/99.3	69.9/97.5
DGM	73.8/99.9	61.0/91.1	31.5/97.3	83.6/99.9	94.8/99.4	68.9/97.6

Table 18
Adversarial training with SPTS against AAT. The attack in adversarial training is MI-FGSM, and the surrogate model is MLP-s.

Attack	MLP-t (%)	CNN-t (%)	ResCNN-t (%)	LSTM-t (%)	Self-attention-t (%)	Average (%)
	IDS18/TON	IDS18/TON	IDS18/TON	IDS18/TON	IDS18/TON	IDS18/TON
MI-FGSM	83.0/99.8	94.6/99.8	94.2/99.8	94.2/99.9	94.2/99.9	92.0/99.8
SIM	94.2/99.8	95.2/99.9	94.9/99.6	94.6/99.9	94.4/99.9	94.7/99.8
VMI-FGSM	83.8/99.9	96.1/99.9	95.5/99.9	94.8/99.9	94.5/99.9	92.9/99.9
DGM	84.8/99.9	97.2/99.9	96.7/99.9	95.8/99.9	95.2/99.9	93.9/99.9

As shown in Table 18, if normal adversarial training allows the model to learn from space from the data points, adversarial training with SPTS narrows the space. This is because only practical AAT needs to be considered in the NIDS field. The experimental results also align with our anticipations: the NIDSs in Table 18 outperform those in Table 17. This also proves that the effectiveness of adversarial training is greatly affected by the quality of the adversarial examples.

Although adversarial training can help NIDSs effectively detect AAT, it still has limitations. First, adversarial training reduces NIDSs' accuracy in normal traffic.[8] Second, adversarial training is unstable, and the model's performance is not always guaranteed, even if the hyperparameters are chosen carefully, as in the case of ResCNN in Table 17. Third, adversarial training is costly, particularly when combined with SPTS. Fourth, normal adversarial training requires a higher quality of data. Otherwise, the results may not be satisfactory.

6. Discussion

In this section, we focus on some limitations regarding STPS and DGM, which will also be the focus of our future research. First, datasets have always been a hot topic within the field of NIDS. An up-to-date and high-quality dataset can help us better evaluate the proposed frameworks and algorithms. Unfortunately, the current publicly available datasets are inadequate and lack benchmarks. On the other hand, different datasets contain different features, making it difficult to migrate directly the SPTS from one dataset to another. For other datasets, SPTS and DGM rely on unique traffic collection and analysis tools that cannot be directly applied. Second, the issue of preserving the functionality of the raw attack traffic. The SPTS modifies the packet length, increasing or decreasing it, which could impact the functionality of the raw attack traffic. However, if the attacker pre-creates two non-functional packets for the traffic, which are used to adjust the maximum and minimum values, the function will be preserved. So, theoretically, SPTS should not affect the functionality of the raw attack traffic. As for the actual result, further experiments are required, which we will continue to investigate in future work. Third, on the interpretability of DGM. It is challenging to quantitatively analyze the relationship between gradient diversity and transferability, mainly due to the black-box nature of DNNs. Currently, we can only explain it intuitively. The generation of AAT can be analogized to the training of DNNs, where the original traffic serves as the "model" to be trained, and the surrogate model serves as the "dataset". The random transformation of the traffic can be viewed as regularization of the "model", thus enhancing the generalization ability of AAT, i.e., the transferability.

[8] Take the CNN-t in Table 18 as an example. Its accuracy is 0.96, precision is 0.99, recall is 0.96, and F1 score is 0.97, lower than the normally trained model.

7. Conclusion

This paper focuses on the generation method of AAT in black-box scenarios and proposes a framework called SPTS. Within the SPTS framework, the attacker can perform attacks without querying the target NIDSs, making the attack process stealthy. SPTS involves feature hierarchization and rectification to produce the practical AAT. We establish the correlations and constraints between features by mathematics. The proposed SPTS is highly extensible, and we implement a variety of adversarial attacks based on it, which are objectively and fairly evaluated. In addition, inspired by model augmentation, we find that enhancing the gradient diversity can further improve the transferability of AAT and propose the DGM. Specifically, DGM performs random transformations to the inputs to produce more robust gradients. To evaluate the effectiveness of SPTS and DGM, we conduct experiments on various DL-based NIDSs. The AAT is mapped to practical packets to evaluate its transferability. We also conduct ablation experiments to investigate the effect of key hyperparameters on performance. In addition, we provide two defense methods to

mitigate SPTS and DGM: normal adversarial training and adversarial training with SPTS. The experimental results demonstrate that although adversarial training can effectively defend against adversarial attacks, it still has some limitations, including a decline in model performance, unstable training, increased cost, and requirements for data quality. Some limitations on SPTS and DGM are also discussed. Finally, we hope this work can inspire more studies in the future.

CRediT authorship contribution statement

Zhongshu Mao: Writing – review & editing, Writing – original draft, Software, Methodology. **Yiqin Lu:** Writing – review & editing, Supervision. **Zhe Cheng:** Writing – original draft, Data curation. **Kaiqiong Chen:** Validation, Data curation.

Declaration of generative AI and AI-assisted technologies in the writing process

During the preparation of this work, we used ChatGPT and Grammarly in order to improve language and readability. After using this tool/service, we reviewed and edited the content as needed and took full responsibility for the content of the publication.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

This work is supported by the Research and Development Program in Key Areas of Guangdong Province, China (No. 2019B010137001).

Data availability

The code will be made available upon request.

References

Abou Khamis, R., Matrawy, A., 2020. Evaluation of adversarial training on different types of neural networks in deep learning-based idss. In: 2020 International Symposium on Networks, Computers and Communications. ISNCC, IEEE, pp. 1–6.

Adeke, J.M., Liu, G., Amoah, L., Nwali, O.J., 2025. Investigating the impact of feature selection on adversarial transferability in intrusion detection system. Comput. Secur. 151, 104327.

Alhajjar, E., Maxwell, P., Bastian, N., 2021. Adversarial machine learning in network intrusion detection systems. Expert Syst. Appl. 186, 115782.

Alhussien, N., Aleroud, A., Melhem, A., Khamaiseh, S.Y., 2024. Constraining adversarial attacks on network intrusion detection systems: transferability and defense analysis. IEEE Trans. Netw. Serv. Manag..

Alsaedi, A., Moustafa, N., Tari, Z., Mahmood, A., Anwar, A., 2020. TON_IoT telemetry dataset: A new generation dataset of IoT and IIoT for data-driven intrusion detection systems. Ieee Access 8, 165130–165150.

Apruzzese, G., Colajanni, M., Marchetti, M., 2019. Evaluating the effectiveness of adversarial attacks against botnet detectors. In: 2019 IEEE 18th International Symposium on Network Computing and Applications. NCA, IEEE, pp. 1–8.

Chen, M., Li, W., Fortino, G., Hao, Y., Hu, L., Humar, I., 2019. A dynamic service migration mechanism in edge cognitive computing. ACM Trans. Internet Technol. (TOIT) 19 (2), 1–15.

Debicha, I., Cochez, B., Kenaza, T., Debatty, T., Dricot, J.-M., Mees, W., 2023. Adv-Bot: Realistic adversarial botnet attacks against network intrusion detection systems. Comput. Secur. 129, 103176.

Dong, Y., Liao, F., Pang, T., Su, H., Zhu, J., Hu, X., Li, J., 2018. Boosting adversarial attacks with momentum. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. pp. 9185–9193.

Dong, Y., Pang, T., Su, H., Zhu, J., 2019. Evading defenses to transferable adversarial examples by translation-invariant attacks. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. pp. 4312–4321.

Du, W., Xue, J., Yang, X., Guo, W., Gu, D., Han, W., 2025. TransfficFormer: A novel transformer-based framework to generate evasive malicious traffic. Knowl.-Based Syst. 113546.

Faker, O., Dogdu, E., 2019. Intrusion detection using big data and deep learning techniques. In: Proceedings of the 2019 ACM Southeast Conference. pp. 86–93.

Frustaci, M., Pace, P., Aloi, G., Fortino, G., 2017. Evaluating critical security issues of the IoT world: Present and future challenges. IEEE Internet Things J. 5 (4), 2483–2495.

Goodfellow, I.J., Shlens, J., Szegedy, C., 2014. Explaining and harnessing adversarial examples. arXiv preprint arXiv:1412.6572.

Han, D., Wang, Z., Zhong, Y., Chen, W., Yang, J., Lu, S., Shi, X., Yin, X., 2021. Evaluating and improving adversarial robustness of machine learning-based network intrusion detectors. IEEE J. Sel. Areas Commun. 39 (8), 2632–2647.

Hashemi, M.J., Cusack, G., Keller, E., 2019. Towards evaluation of nidss in adversarial setting. In: Proceedings of the 3rd ACM CoNEXT Workshop on Big DAta, Machine Learning and Artificial Intelligence for Data Communication Networks. pp. 14–21.

He, K., Kim, D.D., Asghar, M.R., 2023. Adversarial machine learning for network intrusion detection systems: a comprehensive survey. IEEE Commun. Surv. Tutor..

He, K., Zhang, X., Ren, S., Sun, J., 2016. Deep residual learning for image recognition. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. pp. 770–778.

Hinton, G.E., Srivastava, N., Krizhevsky, A., Sutskever, I., Salakhutdinov, R.R., 2012. Improving neural networks by preventing co-adaptation of feature detectors. arXiv preprint arXiv:1207.0580.

Hore, S., Ghadermazi, J., Paudel, D., Shah, A., Das, T., Bastian, N., 2025. Deep packgen: A deep reinforcement learning framework for adversarial network packet generation. ACM Trans. Priv. Secur. 28 (2), 1–33.

Ibitoye, O., Shafiq, O., Matrawy, A., 2019. Analyzing adversarial attacks against deep learning for intrusion detection in IoT networks. In: 2019 IEEE Global Communications Conference. GLOBECOM, IEEE, pp. 1–6.

Jang, D., Son, S., Kim, D.-S., 2022. Strengthening the transferability of adversarial examples using advanced looking ahead and self-cutmix. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. pp. 148–155.

Jmila, H., Khedher, M.I., 2022. Adversarial machine learning for network intrusion detection: A comparative study. Comput. Netw. 214, 109073.

Keegan, N., Ji, S.-Y., Chaudhary, A., Concolato, C., Yu, B., Jeong, D.H., 2016. A survey of cloud-based network intrusion detection analysis. Hum.- Centric Comput. Inf. Sci. 6 (1), 1–16.

Kuppa, A., Grzonkowski, S., Asghar, M.R., Le-Khac, N.-A., 2019. Black box attacks on deep anomaly detectors. In: Proceedings of the 14th International Conference on Availability, Reliability and Security. pp. 1–10.

Lin, Z., Shi, Y., Xue, Z., 2022. Idsgan: Generative adversarial networks for attack generation against intrusion detection. In: Pacific-Asia Conference on Knowledge Discovery and Data Mining. Springer, pp. 79–91.

Lin, J., Song, C., He, K., Wang, L., Hopcroft, J.E., 2019. Nesterov accelerated gradient and scale invariance for adversarial attacks. arXiv preprint arXiv:1908.06281.

Liu, Y., Chen, X., Liu, C., Song, D., 2017. Delving into transferable adversarial examples and black-box attacks. In: 5th International Conference on Learning Representations, ICLR 2017, Toulon, France, April 24-26, 2017, Conference Track Proceedings. OpenReview.net, URL https://openreview.net/forum?id=Sys6GJqxl.

Liu, Z., Ye, D., Tang, L., Zhang, Y., Deng, J., Kuang, W., 2025. TEAM: Temporal adversarial examples attack model against network intrusion detection system applied to RNN. IEEE Trans. Netw. Sci. Eng..

Lu, H., Liu, J., Peng, J., Lu, J., 2025. Adversarial attacks based on time-series features for traffic detection. Comput. Secur. 148, 104175.

Mohammadian, H., Ghorbani, A.A., Lashkari, A.H., 2023. A gradient-based approach for adversarial attack on deep learning-based network intrusion detection systems. Appl. Soft Comput. 137, 110173.

Okada, S., Jmila, H., Akashi, K., Mitsunaga, T., Sekiya, Y., Takase, H., Blanc, G., Nakamura, H., 2025. Xai-driven black-box adversarial attacks on network intrusion detectors. Int. J. Inf. Secur. 24 (3), 1–15.

Qiu, H., Dong, T., Zhang, T., Lu, J., Memmi, G., Qiu, M., 2020. Adversarial attacks against network intrusion detection in IoT systems. IEEE Internet Things J. 8 (13), 10327–10335.

Roshan, K., Zafar, A., 2024. Black-box adversarial transferability: An empirical study in cybersecurity perspective. Comput. Secur. 141, 103853.

Sharafaldin, I., Lashkari, A.H., Ghorbani, A.A., 2018. Toward generating a new intrusion detection dataset and intrusion traffic characterization. ICISSp 1, 108–116.

Sharon, Y., Berend, D., Liu, Y., Shabtai, A., Elovici, Y., 2022. Tantra: Timing-based adversarial network traffic reshaping attack. IEEE Trans. Inf. Forensics Secur. 17, 3225–3237.

Sun, P., Yun, X., Li, S., Yin, T., Si, C., Xie, J., 2025. Advtg: An adversarial traffic generation framework to deceive DL-based malicious traffic detection models. In: Proceedings of the ACM on Web Conference 2025. pp. 3147–3159.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, Ł., Polosukhin, I., 2017. Attention is all you need. Adv. Neural Inf. Process. Syst. 30.

Wang, Z., 2018. Deep learning-based intrusion detection with adversaries. IEEE Access 6, 38367–38384.

Wang, Z., Guo, H., Zhang, Z., Liu, W., Qin, Z., Ren, K., 2021. Feature importance-aware transferable adversarial attacks. In: Proceedings of the IEEE/CVF International Conference on Computer Vision. pp. 7639–7648.

Wang, X., He, K., 2021. Enhancing the transferability of adversarial attacks through variance tuning. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. pp. 1924–1933.

Wang, M., Yang, N., Forcade-Perkins, N.J., Weng, N., 2024. ProGen: Projection-based adversarial attack generation against network intrusion detection. IEEE Trans. Inf. Forensics Secur..

Warzyński, A., Kołaczek, G., 2018. Intrusion detection systems vulnerability on adversarial examples. In: 2018 Innovations in Intelligent Systems and Applications. INISTA, IEEE, pp. 1–4.

Xie, C., Zhang, Z., Zhou, Y., Bai, S., Wang, J., Ren, Z., Yuille, A.L., 2019. Improving transferability of adversarial examples with input diversity. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. pp. 2730–2739.

Yan, H., Li, X., Zhang, W., Wang, R., Li, H., Zhao, X., Li, F., Lin, X., 2023. Automatic evasion of machine learning-based network intrusion detection systems. IEEE Trans. Dependable Secur. Comput..

Yang, K., Liu, J., Zhang, C., Fang, Y., 2018. Adversarial examples against the deep learning based network intrusion detection systems. In: MILCOM 2018-2018 Ieee Military Communications Conference. MILCOM, IEEE, pp. 559–564.

Yao, W., Peng, H., Li, Q., Shen, X., 2025. Modeling realistic adversarial traffic against deep learning-based intrusion detection system in industrial IoT. IEEE Internet Things J..

Zhang, Y., Chen, X., Guo, D., Song, M., Teng, Y., Wang, X., 2019a. PCCN: parallel cross convolutional neural network for abnormal network traffic flows detection in multi-class imbalanced network traffic flows. IEEE Access 7, 119904–119916.

Zhang, Y., Chen, X., Jin, L., Wang, X., Guo, D., 2019b. Network intrusion detection: Based on deep hierarchical network and original flow data. IEEE Access 7, 37004–37016.

Zhang, C., Costa-Perez, X., Patras, P., 2022. Adversarial attacks against deep learning-based network intrusion detection systems and defense mechanisms. IEEE/ACM Trans. Netw. 30 (3), 1294–1311.

Zhang, H., Han, D., Zhuang, S., Wang, Z., Sun, J., Liu, Y., Liu, J., Dong, J., 2025. Explainable and transferable adversarial attack for ml-based network intrusion detectors. IEEE Trans. Dependable Secur. Comput..

Zhang, Y., Wu, Y., Huang, X., 2024. Toward transferable adversarial attacks against autoencoder-based network intrusion detectors. IEEE Trans. Ind. Inform..

Zhang, S., Xie, X., Xu, Y., 2020. A brute-force black-box method to attack machine learning-based systems in cybersecurity. IEEE Access 8, 128250–128263.